
The Language of Complaints

A Big Data Field Manual

NLP-Powered NYC 311 Service Request Triage
and Resolution Time Prediction

George Gideon Sale (gs4602)
Aayush Prnanav Chandrashekar (ac11929)
Shreeram Sankar (ss18731)

CSGY-6513 Big Data, Section D, Term 2
NYU Tandon School of Engineering
Professor Amit Patel

Spring 2026

Source: [https://github.com/george-gideon-S/
cs-gy-6513-big-data-311-nlp](https://github.com/george-gideon-S/cs-gy-6513-big-data-311-nlp)

Preface

This book is an attempt to write down the entire CSKY-6513 final project as a single self-contained reference. Most of what gets written about a class project is either too short (a four-page report) or too sprawling (a Jupyter notebook with 85 cells and no narrative). Neither shape is a good way to teach the project to someone who was not in the room while we built it. This volume splits the difference: it is long enough to cover every piece of the system in depth, and it is structured enough that you can read it linearly or jump to whatever chapter you need.

The system itself is a natural language processing pipeline over the New York City 311 service-request corpus. The corpus is roughly 43 million rows and 13 GB of raw CSV across two SODA endpoints. We sampled 10 million rows and built four things on top of it: a TF-IDF plus logistic regression classifier that predicts the official complaint category from short descriptor text at macro-F1 0.9594; a linear regressor on log1p resolution hours that lifts mean absolute error by 4.1 percent over a strong median-per-category baseline (and by 12 percent on Heat/Hot Water specifically); a Word2Vec plus K-Means clustering view that surfaces operational themes the official taxonomy does not encode; and a pretrained sentence-transformers MiniLM embedding view used in zero-training mode. The whole thing is wrapped in a Streamlit dashboard with six tabs, deployed to Streamlit Community Cloud against an under-1-GB bundle of portable artifacts.

The book is organized in nine chapters plus a glossary, references, and an index.

Chapter 1 (Origin) tells the story of how the project came to be: the CSKY-6513 rubric, why we picked 311, and the thirteen pivots that turned the original proposal into the system we shipped. Chapter 2 (Foundations) is short primers on the pieces of math and engineering you need to know to read the rest: TF-IDF, multinomial logistic regression, log1p regression, Word2Vec, K-Means and silhouette, Sentence-BERT and MiniLM, Spark MLlib pipelines, Spark Structured Streaming, Parquet, Delta Lake, and MurmurHash3. Each primer is a page or two at the depth needed for the project, not encyclopedic. Chapter 3 (Architecture) describes the Lambda-style batch-plus-serve-light pattern, the 1 GB Streamlit Cloud constraint that forced the portability split, and the parallel Spark-native demonstration track. Chapter 4 (Implementation walkthrough) is the longest chapter: phase by phase, with code excerpts, failure stories, and result numbers. Chapter 5 (Results) puts every measured number together with its provenance, in tables and figures. Chapter 6 (Engineering deep dives) zooms in on the gnarlier engineering pieces: per-page parquet checkpoints in the SODA ingest, JVM stale-state recovery in the Spark setup, the gracefully-no-op auto-commit, the baked-in fallback design in the dashboard, and the mmh3 hash bridge. Chapter 7 (Bias and limitations) is candid about what the pipeline cannot do, anchored to Kontokosta and Hong (2021) on 311 reporting bias. Chapter 8 (The deployed dashboard) walks through the six tabs of the user-facing demo, with screenshots. Chapter 9 (Reflections and lessons) closes with what we learned and what we would do next.

If you are reading this as a teammate handed the project to maintain, start with Chapter 3 to get the architecture, then jump to Chapter 4 for whichever phase you are working on. If you are reading this as a course peer trying to understand the project for grading, read Chapters 1, 5, and 9 in that order; the rest is reference. If you are reading this because you are us, six months from now, trying to remember what we did, sorry for any prose that is more clever than informative.

George Gideon Sale, on behalf of the team

May 2026

Contents

Preface	iii
1 Origin	1
1.1 The course rubric we were graded against	1
1.2 Why NYC 311	1
1.3 The original proposal and the gap to reality	2
1.4 Closing reflection: pivots are how engineering meets reality	7
2 Foundations	9
2.1 TF-IDF	9
2.2 Multinomial logistic regression	10
2.3 Log1p regression for heavy-tailed targets	10
2.4 Word2Vec skip-gram	11
2.5 K-Means and silhouette score	11
2.6 Sentence-BERT and MiniLM	12
2.7 Spark MLlib and ML Pipelines	12
2.8 Spark Structured Streaming	13
2.9 Parquet	14
2.10 Delta Lake	14
2.11 MurmurHash3	14
3 Architecture	17
3.1 Lambda-style: batch plus serve-light	17
3.2 The 1 GB Streamlit Cloud constraint	18
3.3 Pure-Python inference path	18
3.4 Parallel Spark-native track	18
3.5 Architecture diagram	19
3.6 Build vs buy	20
4 Implementation Walkthrough	23
4.1 Phase 0: Environment setup	23
4.2 Phase 1: SODA ingest	25
4.3 Phase 2: Preprocessing and label canonicalization	27
4.4 Phase 3: Classifier	28
4.5 Phase 4: Resolution-time regressor	29
4.6 Phase 5: Word2Vec and KMeans	31
4.7 Phase 6: Borough fingerprints	32
4.8 Phase 7: Streamlit dashboard	33

Contents

4.9	Phase 8: Spark Structured Streaming PoC	34
4.10	Phase 9: LDA topic modeling	35
4.11	Phase 10: BERT MiniLM comparison	36
5	Results	39
5.1	Classifier headline	39
5.2	Per-class precision, recall, F1	40
5.3	Regressor headline	40
5.4	Per-category regressor lift	41
5.5	Word2Vec silhouette by k	42
5.6	Word2Vec cluster top terms	43
5.7	BERT vs Word2Vec silhouette comparison	44
5.8	BERT cluster category breakdown	45
5.9	Borough volume	45
5.10	Borough fingerprint terms (full)	46
5.11	The honest MiniLM probe finding	47
5.12	Closing reflection on the result numbers	48
6	Engineering Deep Dives	51
6.1	Per-page parquet checkpoints in the SODA ingest	51
6.2	JVM stale-state recovery in spark_setup	52
6.3	Per-phase resume gates	53
6.4	Mid-phase Spark model checkpoints	53
6.5	Auto-commit graceful no-op	54
6.6	Baked-in fallback in dashboard.baked_in_data	55
6.7	The mmh3 hash bridge (Murmurhash3 vs Python hash)	56
7	Bias and Limitations	59
7.1	Reporting bias as a measurement instrument	59
7.2	Noise - Street/Sidewalk F1 = 0.369 as the prime example	59
7.3	Descriptor-as-dictionary-lookup limits the ceiling	60
7.4	SLA-bounded resolution-time categories	60
7.5	Sample-vs-corpus tradeoff	61
7.6	The MurmurHash3 vs Python hash bridge	61
7.7	Closing reflection on bias and limitations	62
8	The Deployed Dashboard	63
8.1	Triage Bot	63
8.2	City Pulse	64
8.3	Cluster Atlas	64
8.4	Bias Audit	65
8.5	Live Pulse	66
8.6	Pipeline Status	66
8.7	Closing reflection on the dashboard	67

9	Reflections and Lessons	69
9.1	Architecture lesson: portability beats integration	69
9.2	Modeling lesson: keyword baselines first	69
9.3	Honesty about scale: “10M sample, 1.94M cleaned”	70
9.4	Embedding strategy: complementary, not competing	70
9.5	Honest failure: surface, do not hide	70
9.6	Equity: text alone has bounded ceiling	71
9.7	What we would do with more time	71
9.8	What generalized beyond 311	72
9.9	Closing	72
	Glossary	75
	References	79

List of Figures

3.1	Two-tier architecture with the parallel Spark-native track on the right. Heavy distributed training happens on Colab GPU; the deployed dashboard loads slim portable artifacts at startup, so it runs anywhere Python runs. The Spark-native track ships alongside the deploy path for the report and for local demos.	20
5.1	Confusion matrix on the held-out 20 percent split for the top-20 classifier. Diagonal dominance is clear: most classes sit at well above 0.95 F1. The single substantive off-diagonal block is the Noise - Street/Sidewalk row leaking into the Noise - Residential column.	41
5.2	Per-category MAE lift over the median-per-category baseline, sorted from highest lift (top, Heat/Hot Water at +12 percent) to lowest (bottom). The high-variance categories where descriptor text encodes resolution-time signal sit at the top; SLA-bounded categories sit near zero or negative.	43
5.3	Word2Vec + K-Means silhouette by k . Peak is $k = 25$ at silhouette 0.4635; the curve drops slightly at $k = 30$ as the additional cluster splits coherent groups apart.	44
5.4	Silhouette score as a function of cluster count k for both embedding spaces. Pretrained MiniLM reaches 0.6881 at $k = 30$ vs Word2Vec's 0.4551, a 51 percent relative gain with no domain training.	46
5.5	Per-borough 311 complaint volume on the 10M sample. Brooklyn and Queens dominate; Staten Island contributes the smallest share.	48
8.1	The Triage Bot tab. Input text on the left, borough selector on the right, results below. Top result is a big purple card with the predicted category, confidence, and runner-up. Below that, top-5 confidences as horizontal bars, then borough fingerprint terms.	63
8.2	The City Pulse tab. Top-line metrics, choropleth of complaint volume per borough, and per-borough top-categories breakdown.	64
8.3	The Cluster Atlas tab. BERT vs Word2Vec silhouette comparison on top; per-cluster top-terms table on bottom.	65
8.4	The Bias Audit tab. Headline metrics on top; per-class precision/recall/F1 table with WEAK/STRONG flags; confusion matrix below.	65
8.5	The Live Pulse tab. Slider for fetch count, button to trigger the pull, results table with timestamp, original text, predicted category, confidence.	66

List of Tables

3.1	Big Data layer choices, with a one-sentence reason per layer.	21
5.1	Classifier headline metrics on the held-out 20 percent split, top 20 categories. Train pool 1,349,112; train 1,079,326; test 269,786. Trained in 29.7 seconds on Colab Pro A100.	39
5.2	Per-class precision, recall, F1, and support. Flag = WEAK if F1 < 0.85, else STRONG. The Noise - Street/Sidewalk WEAK is the prime example of reporting bias in our model: the descriptor vocabulary is genuinely indistinguishable from Noise - Residential, and the call-taker's choice of one official label over the other is not recoverable from the descriptor alone. . .	40
5.3	Resolution-time regressor MAE in hours and lift over the median-per-category baseline. v2 adds the canonicalized category one-hot. Train 1,008,547; test 251,932. Trained in 12.1 seconds.	42
5.4	Per-category MAE breakdown in hours, model vs median baseline. Positive lift means the model beats the baseline.	42
5.5	Silhouette score by cluster count k for Word2Vec + K-Means.	44
5.6	Selected Word2Vec + K-Means clusters at $k = 25$. Top distinctive tokens per cluster (in decreasing within-cluster frequency).	45
5.7	Silhouette score by cluster count k , Word2Vec vs pretrained MiniLM. Higher is better.	45
5.8	Selected BERT MiniLM clusters at $k = 30$. Showing top categories per cluster with row counts and percentages.	47
5.9	Per-borough 311 complaint volume on the 10M sample (1.59M aggregated rows after empty-token filter).	47
5.10	Top 15 distinctive vocabulary terms per borough by TF-IDF lift. Lift > 1 means the term appears more often in that borough than the city baseline. Filter: at least 100 occurrences in the borough.	49

Chapter 1

Origin

1.1 The course rubric we were graded against

CSGY-6513 Big Data is a project-heavy graduate course in NYU Tandon's computer science department. The Section D, Term 2 final project rubric, as Prof. Amit Patel laid it out at the start of the semester, broke down roughly as follows. A live demonstration in front of the class was worth 33 percent of the project grade. The rest split between code quality (well-organized, readable, documented), the question-and-answer session that followed the live demo (where we had to defend our design choices on the fly), the slide-deck presentation, and the final code submission with a written report.

The structure of the grading mattered a lot for what we built. A live demo that is 33 percent of the grade is a hard constraint: whatever we built had to actually run, in front of an audience, on something we could spin up at lecture time. The textbook-style rubric that gives almost no weight to the live demo would have let us ship a static report and a Jupyter notebook and call it done. This rubric did not. The live demo demanded a deployed surface, ideally something with a public URL we could share so the grader could come back to it after class and click around.

The Q&A session also mattered. A 5-minute Q&A with Prof. Patel after the demo means at least three or four sharp questions about why we chose what we chose. We could not get away with hand-waving on the Big Data parts (Spark vs Pandas? why these features? what about scale?), and we could not get away with hand-waving on the NLP parts either (TF-IDF vs BERT? what's your baseline? what does macro-F1 really tell you?). The whole project had to be defensible end-to-end.

Code quality and the written report were the long-tail components. They mattered for clearing the bar but they would not be the differentiator; the demo and the Q&A would be.

1.2 Why NYC 311

The course let teams pick their own dataset within the Big Data brief. The constraints were that the data had to be genuinely large (greater than a few million rows), publicly available (no paid APIs, no scraping behind logins), and amenable to a Big Data pipeline (not just a pretrained model with a thin wrapper).

We considered several candidates: Reddit comments via Pushshift, NYC Yellow Taxi trips, NYC subway turnstile data, GitHub commit logs, USDA SNAP data. Each one had something against it. Reddit had the scale but the domain was too soft for a defensible model

1. Origin

(sentiment analysis on Reddit comments is a problem with no hard ground truth). The taxi trips had the scale but were essentially a tabular regression problem, not an NLP problem, and our team strength was in NLP. The subway turnstile data had no free text. GitHub commits had free text but the corpus was too noisy and the categories too fuzzy. SNAP data had structure but was too small.

NYC 311 ticked every box. The official New York City 311 system is the city's non-emergency complaint channel: residents call or submit online when they have a heating outage, a noise complaint, an illegal parking issue, a broken streetlight, a rat sighting, a fallen tree branch. City agencies route, triage, and resolve these complaints, and every ticket lands in a public dataset with a free-text descriptor, a structured category, and timestamps. The dataset is one of the largest open municipal corpora in the world.

Two SODA endpoints make up the corpus. The 2020-onward endpoint, identified as `erm2-nwe9`, holds approximately 20 million rows and is the live channel that gets new tickets every day. The 2010-2019 historical endpoint, `76ig-c548`, holds approximately 23 million rows and is frozen in place. Together the corpus is roughly 43 million rows and 13 GB of raw CSV. The schemas overlap but disagree on column names and label casing, which is itself a small data engineering puzzle to solve before any modeling.

Three operational questions motivated the work. First, given the short free-text descriptor a resident or call-taker types, can we predict the official complaint category with high accuracy? This is the bread-and-butter classification question and it underpins downstream routing. Second, given the same text plus structured covariates, can we predict how long the request will take to close, beating the simple median-per-category heuristic that already reflects city service-level agreements? This is the regression question and it speaks to capacity planning. Third, can we discover latent issue groupings that the official taxonomy misses, by clustering text embeddings and surfacing distinctive vocabulary by geography? This is the unsupervised-discovery question and it points at the kind of operational signal a static taxonomy will always miss.

1.3 The original proposal and the gap to reality

The first proposal was idealistic. We told Prof. Patel we would pull the full 43 million rows, build classifiers and regressors and topic models, deploy a live dashboard backed by Spark Structured Streaming, partition the city into 71 community districts for fine-grained geographic analysis, and ship the whole thing on Databricks Free Edition. The plan had everything except a realistic estimate of how much trouble each piece would give us.

The next four months turned that proposal into thirteen separate stories of why we changed our mind on a piece. Each one is named below and explained in detail in the rest of this chapter. The stories are in the order they hit, as best we can reconstruct from the git history.

Story 1: Plotly Dash to Streamlit

The proposal said Plotly Dash. Dash is more flexible than Streamlit but it is also more verbose: every component is a callback and every layout is a tree of `html.Div` objects. With three of us splitting the build, we needed something where one person could iterate fast enough to hand off polished tabs to the other two.

Streamlit's iteration loop is much shorter. A tab is a Python module that calls `st.header`, `st.button`, `st.plotly_chart` in sequence; reading the file top to bottom is the same

as reading the user flow. There are no callbacks to wire up, no state machines to debug. The deploy story is also better: Streamlit Community Cloud is one click from a public GitHub repo, no Docker image to maintain, no separate CI to set up.

We pivoted to Streamlit on day three. The repo was scaffolded around `dashboard/app.py` and a folder of per-tab modules. Two months later, when the same Streamlit Cloud free-tier 1 GB constraint became the architectural pivot point in Chapter 3, we did not regret the choice; if anything, the constraint forced a cleaner build.

Story 2: Databricks Free Edition to Google Colab Pro

The proposal said Databricks Free Edition. The Free tier comes with a managed Spark cluster and one Delta Lake catalog, which sounded perfect for the scale. The reality on day five was that Databricks Free has a hard cap on cluster runtime per session, plus a cap on storage that we would burn through after one full SODA pull, plus an absence of a public-shareable demo URL. The dashboard would have to be re-deployed on Streamlit Cloud anyway, but we would lose the ability to run a Colab-style notebook against the Databricks cluster without setting up a private endpoint.

Google Colab Pro turned out to be the better tool. Colab Pro gives us an A100 GPU on demand, 50 GB of high-RAM, a free Google Drive mount for persistence, and a notebook URL that anyone with the link can open. Spark runs in `local[*]` mode against the GPU node, which is fine at our scale: 10 million rows fits in driver memory after columnar compression, and Spark MLlib’s distributed L-BFGS uses the GPU node’s cores rather than a real cluster.

We pivoted to Colab Pro on day five and never looked back. The auto-commit cell at the end of each phase pushes the portable artifacts back to GitHub via a fine-grained Personal Access Token, which is the bridge from Colab to the deployed Streamlit app.

Story 3: 43M full corpus to 10M sample

The proposal said full 43M. The intuition was that more data is always better, that Spark could handle it, and that we should not under-sample on principle.

This was wrong on each axis. Spark in `local[*]` mode on a 50 GB high-RAM Colab node can handle 10M rows comfortably and 43M rows uncomfortably. The TF-IDF stage’s HashingTF is memory-bound, not compute-bound, and a 43M-row hash with a 16K bucket size lands a 16 KB-per-row footprint into memory before the IDF reduction. At 43M rows that is 700 GB intermediate. With aggressive Parquet checkpointing we could have driven it down, but the wall-clock cost was prohibitive: a single Phase 3 training run would have stretched past the Colab Pro session timeout.

The 10M sample was the right compromise. We pulled 5M from each endpoint, stratified by canonicalized category so the minority classes were not lost in the sample. The combined sample was approximately 1.94 million rows after the empty-token filter we apply in Phase 2 (descriptors with zero surviving tokens after preprocessing get dropped). This is the working corpus for everything in this book; when the dashboard says “10M sample” it means the upstream sample that became 1.94M after cleaning.

Story 4: 71 community districts to 5 boroughs

The proposal said community districts. The `mzpm-a6vd` dataset on the NYC OpenData portal returns a GeoJSON of 71 community-district polygons, which would have given us geographic resolution finer than a borough but coarser than a ZIP code. We spec'd Phase 6 around this dataset.

When we tried to download it on week three of the build, the endpoint returned features with null geometries. Some kind of upstream issue on the city's side; we filed a question on the OpenData forum and never got a response. The geometries we needed for spatial joins simply did not arrive.

We pivoted to the five-borough boundary file. The borough column already exists on every 311 ticket as a string (MANHATTAN, BROOKLYN, etc.), so no spatial join is needed: a string match suffices. The five-borough GeoJSON is a 5-polygon file we ship in the repo at `dashboard/assets/nyc_boroughs.geojson`, total file size around 50 KB. The trade-off is some loss of geographic resolution; the upside is that boroughs already produce interpretable fingerprints, as Chapter 5 shows in detail.

Story 5: GBTRegressor to LinearRegression on log1p

The proposal said gradient-boosted trees for resolution-time regression. The intuition was that resolution times have non-linear relationships with category, agency, and time-of-day, and a tree-based model would find those relationships automatically.

The reality was that resolution time has a heavy-tailed distribution: most tickets close in hours, some take days, some take months. A linear model on the raw target gets dragged around by the tail. A linear model on $\log_{1p}(\text{hours})$ where $\log_{1p}(x) = \log(1 + x)$ is a much better fit. The log transform compresses the tail, the linear model finds the pattern, and the inverse-transform `expm1` brings predictions back to hours. On our data the linear-on-log1p model and a GBTRegressor reached the same MAE within 0.2 percent. The linear model trained 100x faster and produced coefficients we could ship as a numpy artifact.

We pivoted to LinearRegression on log1p in Phase 4. The v2 variant adds the predicted canonical category one-hot as a feature, which simulates the realistic operational pipeline where the classifier output feeds the regressor input.

Story 6: Python hash() to mmh3 with seed=42

This is the bug-fix story we are most proud of finding. Spark MLlib's HashingTF uses MurmurHash3_x86_32 internally with a fixed seed of 42. When we exported the trained classifier as a portable numpy artifact for the deployed Streamlit app, the inference code re-tokenized the input and hashed each token into the same vocabulary space. The hashing function we used was Python's built-in `hash()`.

For a few days the deployed Triage Bot tab was returning the class prior on every input. Type "my heat is out" and the predicted class would be "Noise - Residential". Type "loud music upstairs" and the predicted class would also be "Noise - Residential". The probabilities were nearly the marginal distribution of the training set. We had no idea why.

The fix was small but the diagnosis took half a day. Python's built-in `hash()` is randomized per-process via the `PYTHONHASHSEED` environment variable, which is set to a random value at Python startup unless explicitly overridden. So a token that landed at hash bucket 1234 during Spark training would land at some random other bucket at deploy-time

inference, and the entire feature vector was effectively random. The model was making predictions on random features, which means predicting the class prior.

The fix is in commit 07745d7. We replaced `hash(token)` with `mmh3.hash(token, seed=42)` from the `mmh3` Python package. The `mmh3` binding is a Python wrapper for the same `MurmurHash3_x86_32` algorithm Spark MLlib uses, with the same seed. Token buckets at deploy time now match training-time buckets bit-for-bit.

Story 7: Spark Structured Streaming PoC to on-demand SODA pull

The proposal said Spark Structured Streaming would back the dashboard’s Live Pulse tab. The plan was to have a watch folder where new CSV files would appear, a streaming query would classify each incoming row, and the dashboard would tail the resulting Delta sink.

This was incompatible with Streamlit Community Cloud’s deploy environment. The free tier ships Python 3.11 with no Java runtime, which means PySpark cannot start a JVM, which means there is no Spark at deploy time. The PoC works fine when run locally on a Colab session or a JVM-equipped local machine, but it cannot be the dashboard’s Live Pulse path.

We pivoted to an on-demand SODA pull. The Live Pulse tab queries the live SODA endpoint for the most recent 311 tickets at request time, runs each through the portable numpy classifier, and renders the results. The streaming PoC still ships in the repo at `src/stream_spark.py` as a runnable demo, separate from the deploy path. Chapter 4 covers both.

Story 8: `numpy<2.0` to `numpy>=1.24`

We had pinned `numpy<2.0` in our Phase 1 setup based on a stale Spark compatibility note. When we deployed the dashboard the first time, the build crashed with “numpy.dtype size changed” from a binary ABI mismatch in pandas. Both Colab and Streamlit Cloud had moved to numpy 2.x and our pin was forcing a downgrade that broke pandas.

The fix in commit 820355c dropped the upper bound. The current pin is just `numpy>=1.24`.

Story 9: Auto-commit hard fail to graceful no-op

The auto-commit pattern is the bridge from Colab notebook output to the deployed dashboard. After each phase, the notebook calls a helper that runs `git add models/portable/` and `git add dashboard/assets/`, commits, and pushes via a Personal Access Token. The push lands the new artifacts in the GitHub repo, and the next time Streamlit Cloud rebuilds the deployed app, it picks them up.

Early in the project we cleaned up the repo so `models/portable/` and `dashboard/assets/` JSON files were gitignored, since they were generated on every Colab run and we did not want them tracked. After that cleanup, `git add` would return exit code 1 with the message “the following paths are ignored by one of your .gitignore files,” and the auto-commit cell would crash the notebook.

The fix in commit fe3ee6b changes `git add` to use `check=False` (so a non-zero exit does not raise), then verifies via `git diff -cached -quiet` whether anything actually got staged, and skips the commit-and-push if nothing was. Later, when we re-allowed the runtime artifacts in commit 24e9c1d so they ship with the deployed app, the same auto-commit code worked unchanged in the new gitignore policy.

Story 10: Spark parquet ancient-date crash

Some 311 rows have `closed_date` timestamps before 1900-01-01. These are data-quality artifacts from the historical 2010-2019 endpoint, but Spark 3.0+ refuses to write timestamps before 1900 to Parquet INT96 unless you set explicit rebase configuration. Without the config, Spark throws a SPARK-31404 error mid-write and corrupts the parquet directory.

The fix in commit `3d559e1` sets four config options on the Spark session: `spark.sql.parquet.int96` the matching `InRead`, and the `datetimeRebaseMode` pair. CORRECTED keeps proleptic-Gregorian semantics, which is the right thing for modern readers; LEGACY would rebase to the old hybrid calendar for Spark 2.x interop, which we do not need.

Story 11: Bare in-memory SODA pull to per-page parquet checkpoints

The first SODA ingest was a tight loop. `requests.get` fetches a page of 50K rows, the page is parsed to a pandas DataFrame, and the frames are accumulated in memory until we have all 5 million rows. If anything went wrong mid-pull, every fetched page was lost.

Things went wrong mid-pull repeatedly. SODA throttles aggressively above 1K requests per hour for anonymous calls and even with an app token there are short windows where requests fail. A network blip on Colab's outbound network kills the pull. A Colab session timeout in the middle of the night drops every accumulated frame.

The fix in commit `8effe34` writes every fetched page to its own parquet part-file under `out_path.partial/part_NNNNN.parquet` before the next `requests.get`. The retry budget is 8 attempts per page with backoff scheduled at 1, 2, 4, 8, 16, 30, 60, 60 seconds (capped at 60 because SODA's throttle window can take that long to clear). On crash, the next run scans the partial directory, loads every part file in order, recovers the offset from the highest part number, and resumes from there. No rows are lost.

Story 12: JVM stale-state recovery

Spark uses a JVM. The JVM lives across cells in a single notebook session via a singleton `SparkSession`. When the JVM crashes (out-of-memory, port conflict, py4j socket drop), the singleton handle still points at the dead process. `SparkSession.builder.getOrCreate()` on the next cell tries to reuse the dead handle and fails with `ConnectionRefusedError`.

The fix in commit `fca1c8e` adds a `_kill_stale_session()` helper at the top of `src/spark_setup.py::get_spark()`. Before building a new session, it explicitly clears `SparkSession._instantiatedSession` and `SparkContext._active_spark_context` swallowing any exceptions because the dead session will throw on `.stop()` too. The next `getOrCreate()` sees no singleton, builds a fresh one, and the rest of the notebook runs.

Story 13: gensim missing from requirements

Phase 5 saves the trained Word2Vec model as a gensim KeyedVectors file (`models/portable/word2vec.k`). The conversion from Spark's Word2VecModel to gensim is a small loop that builds vectors via `gensim.models.KeyedVectors.add_vectors(words, vectors)`.

When the auto-commit cell first ran in Phase 5, the gensim import failed. Gensim was not in our `requirements-train.txt` despite being mentioned in code comments. The fix in commit `195cffd` pinned `gensim>=4.3` and `nltk>=3.8` in the training requirements file. The deploy requirements (`requirements.txt`) deliberately do not include gensim,

since the deployed dashboard does not need it for prediction; the Cluster Atlas tab degrades gracefully when gensim is not installed.

1.4 Closing reflection: pivots are how engineering meets reality

Of the thirteen stories above, none of them is a story of failure. Each one is a story of where the proposal hit a piece of reality and bent. A proposal that does not bend in those places either gets shipped late, gets shipped broken, or does not get shipped. The hardest thing to learn as a graduate engineer is that the proposal is the starting point, not the contract.

The pivots that matter most are the ones that change the architecture, not the ones that swap a library for another library. Story 1 (Dash to Streamlit) was a library swap; the architecture stayed the same. Story 7 (Spark Streaming to SODA pull) was an architecture pivot; the streaming PoC still ships, but it is not the deploy path. Story 4 (71 districts to 5 boroughs) was an architecture pivot; the geography layer simplified by an order of magnitude. Story 11 (in-memory pull to per-page checkpoints) was an architecture pivot; crash safety became a property of the system rather than something we hoped for.

The rest of this book is the system after all thirteen pivots have been absorbed. We will not re-litigate the pivots; we will explain how the system works now. But the pivots are the reason the system works at all, and they are the most useful artifact a class project can leave for the people who come next. This is what it actually looks like to build a Big Data system in a semester, with three students, on free-tier infrastructure, against a public dataset that fights you sometimes.

1. Origin

Chapter 2

Foundations

This chapter is short primers on the math and engineering you need to read the rest of the book. Each primer is a page or two at the depth needed for the project, not encyclopedic. If you are already comfortable with TF-IDF, multinomial logistic regression, log1p targets, Word2Vec, K-Means, Sentence-BERT, Spark MLlib pipelines, Spark Structured Streaming, Parquet, Delta Lake, and MurmurHash3, you can skim this chapter or skip it entirely.

2.1 TF-IDF

TF-IDF stands for Term Frequency-Inverse Document Frequency. It is the canonical bag-of-words feature representation for short text classification, predates modern neural NLP by half a century, and continues to be a strong baseline (often hard-to-beat) on short-text categorical tasks like ours.

The two terms split apart. The *term frequency* of a token t in a document d is the count of times t appears in d :

$$\text{tf}(t, d) = \text{count}(t \in d). \quad (2.1)$$

Some variants normalize by document length, some take the log; the simple count works fine for short descriptors where most documents are 1 to 6 tokens long.

The *inverse document frequency* of t across a corpus D of N documents is a weighting that down-weights tokens that appear in many documents and up-weights tokens that appear in few:

$$\text{idf}(t, D) = \log \frac{N}{1 + |\{d \in D : t \in d\}|}. \quad (2.2)$$

The plus-one in the denominator is a smoothing convention that avoids dividing by zero when a token appears in every document. The base of the log does not matter for downstream classification because the logarithm shifts every column by a constant.

The TF-IDF score is the product:

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D). \quad (2.3)$$

Two implementation notes matter for our pipeline. First, Spark MLlib's `HashingTF` hashes tokens into a fixed-size bucket array using MurmurHash3 (see Section 2.11), which means the feature dimension is a tunable hyperparameter (we use 16,384 for the classifier and 4,096 for the regressor) rather than the actual vocabulary size. Hash collisions occur but they are infrequent enough at our scale that they do not measurably hurt model quality.

Second, the IDF is fit on the training set only and applied to the test set; if you fit IDF on the combined corpus you leak test information into training.

Note. TF-IDF on short descriptors is a remarkably strong baseline because the descriptors are mostly short categorical labels with constrained vocabulary. The keyword-baseline F1 of 0.78 in Chapter 5 is the consequence: the TF-IDF representation already encodes most of the operational signal.

2.2 Multinomial logistic regression

The classifier is multinomial logistic regression with L2 regularization. Logistic regression generalizes the binary classification problem to K classes via the softmax function. Given a feature vector $x \in \mathbb{R}^d$, a coefficient matrix $W \in \mathbb{R}^{K \times d}$, and a bias vector $b \in \mathbb{R}^K$, the predicted class probabilities are

$$P(y = k \mid x) = \frac{\exp(w_k^\top x + b_k)}{\sum_{j=1}^K \exp(w_j^\top x + b_j)}. \quad (2.4)$$

The training objective is the cross-entropy loss summed across the training set, with an L2 penalty $\frac{\lambda}{2} \|W\|_F^2$ to prevent overfitting:

$$L(W, b) = - \sum_{i=1}^N \log P(y_i \mid x_i) + \frac{\lambda}{2} \|W\|_F^2. \quad (2.5)$$

Spark MLlib fits this with distributed L-BFGS, which is a quasi-Newton method that approximates the inverse Hessian using past gradients. L-BFGS is the right tool here because the objective is smooth and convex, so any local minimum is the global minimum.

Why this primer matters for our project: when the deployed classifier was returning the class prior on every input (Story 6 in Chapter 1), the symptom was that the softmax distribution looked like the marginal distribution of y in the training set. If $W^\top x \approx 0$ for all x because x is effectively random, the bias term b dominates the logits, and softmax of b is the marginal class distribution. That is exactly what we saw, and it is what told us the feature path was broken (the Murmurhash bridge was missing) rather than the model itself.

2.3 Log1p regression for heavy-tailed targets

The resolution-time regressor predicts hours-to-close, a quantity with a heavy right tail. Most 311 tickets close within hours, some take days, and a long tail of tickets (Heat/Hot Water during winter outages, complex Plumbing investigations, ongoing Sewer issues) take weeks or months. Fitting a linear model on the raw target h pulls the coefficients toward the tail; the model gets dragged by the slow tickets and underfits the fast ones.

The fix is to fit on a log-transformed target. The natural log $\log(h)$ would work but blows up at $h = 0$ (which happens occasionally in the data: a ticket created and closed in the same minute). The standard fix is the log-plus-one transform:

$$y = \log(1 + h). \quad (2.6)$$

The transform $\log(1 + h)$ is finite at $h = 0$ (it equals 0), monotonic in h , and compresses the tail logarithmically. Predictions in y -space are inverse-transformed via:

$$\hat{h} = \exp(\hat{y}) - 1. \quad (2.7)$$

This pair is sometimes written `log1p` and `expm1` in numerical libraries; the `m1` in `expm1` stands for “minus one,” returning $\exp(x) - 1$ with better numerical precision near zero.

A subtlety: the linear model’s MAE in y -space does not equal the MAE in h -space. The transformation is not affine, so we report MAE on the inverse-transformed predictions in hours. That is the metric the dashboard shows and the metric that compares to the median-per-category baseline.

2.4 Word2Vec skip-gram

Word2Vec (4) is a neural word embedding learned from a co-occurrence objective. The skip-gram variant we use trains an embedding matrix $E \in \mathbb{R}^{|V| \times d}$ where $|V|$ is the vocabulary size and d is the embedding dimension, by predicting context words from a center word. For a center word at position i in a sequence, and a window of $\pm c$ words, the objective is:

$$L = - \sum_i \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{i+j} | w_i). \quad (2.8)$$

The conditional $P(w_{i+j} | w_i)$ is parameterized via dot products of embedding vectors and a softmax over the full vocabulary, but the full softmax is too expensive for large vocabularies. The standard trick is *negative sampling*: instead of computing the softmax denominator over all $|V|$ words, sample k “negative” words at random and train a binary classifier that distinguishes the true context word from the negatives. Spark MLlib’s Word2Vec uses hierarchical softmax instead of negative sampling, which is the equivalent trade-off via a different mechanism.

Document embeddings are built from word embeddings by averaging in-vocabulary word vectors per document. Spark MLlib’s Word2VecModel does this for us when we call `model.transform(df)` with a `tokens` column.

For our project we trained Word2Vec with vector size 100, window 5, and min count 5. The min count filter shrinks the vocabulary to 1,187 distinct words, which reflects the narrow vocabulary of the descriptor field. We export the trained vectors as a gensim KeyedVectors object so the dashboard can load them in pure Python without booting Spark.

2.5 K-Means and silhouette score

K-Means is the canonical centroid-based clustering algorithm. Given n points in $\mathbb{R}^d$ and a target number of clusters k , K-Means finds k centroid points that minimize the sum of squared distances from each data point to its nearest centroid:

$$\min_{C_1, \dots, C_k} \sum_{i=1}^n \min_j \|x_i - C_j\|^2. \quad (2.9)$$

The standard algorithm alternates between (a) assigning each point to its nearest centroid and (b) updating each centroid to the mean of its assigned points, until convergence. The result depends on the initialization; Spark MLlib uses `k-means++` initialization, which spreads the

initial centroids out by sampling them with probability proportional to squared distance from the existing centroids.

The hyperparameter k has to be chosen. There is no universal rule. Two common heuristics: the elbow method (plot intra-cluster sum of squares against k and look for the elbow) and silhouette score. We use silhouette.

The silhouette score for a single point i is:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad (2.10)$$

where $a(i)$ is the mean distance from i to other points in the same cluster, and $b(i)$ is the minimum mean distance from i to points in any other cluster. Silhouette ranges from -1 (point should be in another cluster) to $+1$ (point is well-clustered), with values around 0 indicating a point on the boundary between two clusters. The corpus-level silhouette is the mean of $s(i)$ over all points, and we pick the k with the highest mean silhouette as the operating point.

For our project, Word2Vec + K-Means peaks at $k = 25$ with silhouette 0.4635 ; pretrained MiniLM + K-Means peaks at $k = 30$ with silhouette 0.6881 . Chapter 5 reports the full sweep.

2.6 Sentence-BERT and MiniLM

Sentence-BERT (5) is the canonical adaptation of BERT-style transformer encoders to fixed-dimensional sentence embeddings. The original BERT produces a [CLS]-pooled vector per sentence, but those vectors are not particularly well-suited for sentence-similarity tasks because BERT was trained on token-level objectives (masked language modeling and next-sentence prediction). Sentence-BERT fine-tunes BERT in a siamese architecture on contrastive sentence pairs, producing a checkpoint whose pooled output respects sentence-level cosine similarity.

MiniLM (6) is a family of distilled transformers from Microsoft Research. The MiniLM architecture takes a large BERT teacher and distills it into a smaller student via deep self-attention transfer, where the student’s attention distributions are matched to the teacher’s at every layer. The result is a much smaller model that retains most of the teacher’s quality.

The specific checkpoint we use is `sentence-transformers/all-MiniLM-L6-v2`. It is a 6-layer transformer encoder with about 14 million parameters, hidden size 384, and pooled sentence output of 384 dimensions. We use it without fine-tuning. The encoding step on an A100 GPU takes about 4.3 seconds for 99,999 documents (23,060 docs/sec), which is fast enough that we encode at notebook time rather than in advance.

The headline finding: at matched cluster count $k = 30$, MiniLM produces silhouette 0.6881 vs Word2Vec’s 0.4551 . That is a 0.23 absolute gain, 51 percent relative. The pretrained model with zero domain training beats a from-scratch Word2Vec on global cluster geometry. But Chapter 5 also reports the honest failure mode: MiniLM’s nearest-neighbor retrieval to the query “rat infestation in the building” returns mostly Heat/Hot Water descriptors carrying the literal phrase “ENTIRE BUILDING.”

2.7 Spark MLlib and ML Pipelines

Apache Spark (8) is the distributed-compute substrate underneath everything in this project. Spark MLlib (3) is the machine-learning library that ships with Spark. The two abstractions

in MLlib that matter for our pipeline are *Estimator* and *Transformer*.

An Estimator is anything that can be fit to data. IDF, LogisticRegression, StringIndexer, LinearRegression, Word2Vec, KMeans, LDA are all Estimators. Calling `estimator.fit(df)` returns a fitted model.

A Transformer is anything that takes a DataFrame and returns a DataFrame with one or more new columns added. HashingTF is a Transformer (no parameters to fit). IDFModel, LogisticRegressionModel, KMeansModel are Transformers (the result of fitting their corresponding Estimator). TextPreprocessor (our custom Spark Transformer in `src/preprocess.py`) is also a Transformer.

A *Pipeline* is an ordered list of Estimators and Transformers. Calling `pipeline.fit(df)` fits each Estimator in order, treating earlier-stage outputs as inputs to later-stage Estimators. The result is a *PipelineModel* which is itself a Transformer. PipelineModels can be saved and loaded:

```
1 pipeline = Pipeline(stages=[indexer, hashing_tf, idf, lr])
2 model = pipeline.fit(train_df)
3 model.write().overwrite().save("/path/to/model")
4 loaded = PipelineModel.load("/path/to/model")
5 predictions = loaded.transform(test_df)
```

Saved PipelineModels are directories containing the per-stage state in Parquet format. They cannot be loaded outside Spark; Streamlit Cloud has no Java, so we cannot load them at deploy time. That is why we export portable artifacts in numpy / gensim format alongside the Spark PipelineModel.

2.8 Spark Structured Streaming

Spark Structured Streaming is Spark's API for incremental processing of unbounded data. The core abstraction is that a streaming DataFrame is a DataFrame that grows over time; you write the same DataFrame operations you would write for a batch query, and Spark runs them as a continuous loop over new data.

The execution model is *micro-batches*. By default, Spark groups incoming data into small batches (every 100 ms, every second, every 30 seconds) and runs the query over each batch. The user controls the batch cadence via the `trigger` argument:

- `processingTime("30 seconds")`: every 30 seconds.
- `availableNow=True`: process every input that exists at start time, then stop. This is what we use, because it works on Databricks Free Edition and on Colab.
- `continuous("1 second")`: continuous mode (no micro-batching), only supported for a few sinks.

Exactly-once semantics are guaranteed when the source supports replay (e.g., file source, Kafka) and the sink supports idempotent writes (e.g., file sink with `checkpointLocation`). The checkpoint directory holds offset metadata so a query that crashes mid-batch resumes from the last committed offset on restart.

For our Phase 8 PoC, we readStream from a watch folder of CSV files, run the trained PipelineModel as a Transformer on each batch, and writeStream to a Delta Lake sink with append mode. The full code is in `src/stream_spark.py`.

2.9 Parquet

Apache Parquet is a columnar file format optimized for analytical workloads. Three properties matter for our pipeline.

First, columnar storage means that scans which read only a subset of columns are much faster than equivalent CSV scans. When Spark loads a Parquet file and the query is `SELECT problem_detail FROM corpus`, only the `problem_detail` column's bytes are read off disk; the other 12 columns are not touched.

Second, Parquet supports column-level compression. We use Snappy, which is the default in Spark. Snappy is a fast, mid-compression codec that hits a good point on the speed-vs-size curve. Our 10M-row sample lands at about 1.2 GB on Drive after Snappy.

Third, Parquet supports *predicate pushdown*: a filter like `WHERE created_date > '2023-01-01'` can be evaluated against the file's column statistics (min/max per row group) before the actual data is read, skipping entire row groups whose statistics rule them out. This makes time-window queries very fast.

A gotcha in our pipeline: Parquet stores INT96 timestamps, and Spark 3.0+ requires explicit rebase configuration when writing timestamps before 1900-01-01. Story 10 in Chapter 1 covers the fix.

2.10 Delta Lake

Delta Lake is a transactional layer on top of Parquet. A Delta table is a directory of Parquet files plus a `_delta_log/` subdirectory of JSON transaction logs. The transaction log gives Delta four properties Parquet does not have:

- *ACID transactions*. Concurrent writes are serialized; readers always see a consistent snapshot.
- *Time travel*. A query can be run against an arbitrary historical version of the table by giving Delta a version number or timestamp.
- *Schema evolution*. Columns can be added, dropped, or renamed without rewriting the underlying Parquet.
- *Streaming I/O*. A Delta table can be read as a stream, with downstream queries notified of new appends.

For our project, the Phase 8 streaming PoC writes to a Delta sink. The dashboard's old design tailed the Delta sink for the Live Pulse tab. After the streaming pivot to on-demand SODA pulls, Delta is no longer in the deploy path, but the streaming PoC at `src/stream_spark.py` still uses it.

2.11 MurmurHash3

MurmurHash3 (11) is a non-cryptographic hash function designed by Austin Appleby in 2008. It produces a 32-bit or 128-bit hash with high avalanche properties (small input changes produce uniformly-distributed output changes), is faster than cryptographic hashes (no security guarantees, no rounds of mixing), and is used in many systems for hash-based data structures.

Spark MLlib's HashingTF uses the 32-bit variant, `MurmurHash3_x86_32`, with a fixed seed. The seed is a small integer (default 42 in Spark). The hash function takes a UTF-8 byte sequence and the seed, and returns a signed 32-bit integer. To get a bucket index, the result is taken modulo the bucket count.

Why this primer matters for our project: when the deployed classifier needs to hash a token at inference time, it has to produce the same bucket index that Spark MLlib produced at training time. Python's built-in `hash()` is randomized per-process via `PYTHONHASHSEED`, so it cannot be used. The fix is the `mmh3` package, which is a thin Python binding around the same MurmurHash3 algorithm. Calling `mmh3.hash(token, seed=42)` produces the bit-identical 32-bit signed integer that Spark would produce, and `mmh3.hash(token, seed=42) % vocab_size` produces the matching bucket. Story 6 in Chapter 1 covers the bug; the fix is in commit `07745d7`.

Chapter 3

Architecture

The architecture of the system is shaped by one constraint: the live demo at lecture time has to load fast, render fast, and run on a free-tier cloud service. Streamlit Community Cloud’s free tier limits the deployed environment to roughly 1 GB and provides no Java runtime, so Spark cannot run there. The naive option of shipping the trained Spark MLlib pipeline directly is not available.

This chapter explains the resulting two-tier architecture, the constraints that forced it, and the parallel Spark-native track that ships alongside the deploy path for the report and for any local or Colab demo.

3.1 Lambda-style: batch plus serve-light

The Lambda architecture, in its original form, splits a system into a batch layer that processes large amounts of data slowly and a speed layer that processes small amounts of data quickly, with a serving layer that combines the two. Our system is a flavor of that.

The batch layer is Spark MLlib running on Colab Pro with a high-RAM A100 GPU node. It pulls the SODA endpoints, normalizes schema, preprocesses text, fits a TF-IDF + LR classifier, fits a log1p linear regressor, fits Word2Vec embeddings, sweeps K-Means cluster count, computes per-borough TF-IDF lift, and encodes a 100K subsample with pretrained MiniLM. Every stage produces a Parquet checkpoint on Drive (so phases are resumable) and a portable artifact that the deployed dashboard can consume (numpy npz, gensim kv, JSON).

The serve layer is the Streamlit dashboard at `dashboard/app.py`. It loads the portable artifacts at startup (cached via Streamlit’s `@st.cache_resource` decorator), and serves predictions via pure-Python numpy operations. The TF-IDF + LR classifier is a single matrix multiply plus softmax; the regressor is a dot product plus inverse-log1p; the borough fingerprints are a JSON lookup. The total deploy bundle is around 220 MB, well under the 1 GB free-tier limit, and serves predictions in well under a second.

There is no traditional speed layer in our system. The Live Pulse tab fetches new 311 tickets on demand from the SODA endpoint and runs the same numpy classifier; this is closer to a serving-layer feature than a speed-layer aggregation. The Phase 8 Spark Structured Streaming PoC *is* a speed layer in the Lambda sense, but it is not part of the deployed dashboard for the reasons explained in Story 7 of Chapter 1.

3.2 The 1 GB Streamlit Cloud constraint

Streamlit Community Cloud is a free hosting service for Streamlit apps. The free tier comes with a fixed runtime container (Python 3.11, no Java, no GPU, no Docker control), bound to a GitHub repository, with deploys triggered automatically on push. The free tier ships about 1 GB of disk and 1 GB of RAM at runtime; the Python virtual environment plus all dependencies counts against this budget.

A few corollaries follow from this. First, no Java means no Spark, which means no `PipelineModel.load()`, which means we cannot ship the trained Spark pipeline. Second, no GPU means we cannot run sentence-transformers MiniLM at deploy time; the encoder model is small enough to fit in memory but inference is too slow on CPU for an interactive dashboard. Third, the 1 GB budget means we cannot ship the full 1.94M sample to deploy; we ship per-borough aggregates and per-cluster summaries instead.

This chain of corollaries forces the portability split. The batch layer exports portable artifacts; the serve layer loads them. The artifacts are deliberately slim so the deploy stays under budget.

3.3 Pure-Python inference path

The deployed classifier is a numpy npz file with five fields: `vocab_size` (the bucket count of HashingTF), `idf` (a float vector of length `vocab_size`), `coefs` (a $K \times \text{vocab_size}$ float matrix of LR coefficients), `intercepts` (a length- K float vector of LR biases), and `labels` (a length- K array of class strings).

Inference on a single complaint takes the following steps. First, tokenize and lower-case the input, drop English stopwords, drop tokens shorter than three characters, and lemmatize. Second, hash each surviving token via `mmh3.hash(token, seed=42) % vocab_size` and accumulate counts in a length-`vocab_size` float array (this is the raw TF). Third, multiply elementwise by the IDF vector to get the TF-IDF feature vector. Fourth, compute the logits `coefs @ feature + intercepts` (a single matrix-vector multiply at $K = 20$ classes and `vocab_size = 16,384`, which is fast in numpy). Fifth, softmax the logits to get class probabilities, with a max-subtract trick for numerical stability.

The whole inference path is about 30 lines of numpy. It runs in under 10 milliseconds on a laptop and well under 1 second end-to-end on Streamlit Cloud's CPU container, including the Streamlit rendering overhead.

The regressor inference is even shorter. Load the regressor npz (LR coefficients on the engineered features), look up the predicted category, look up the median resolution time for that category from the baked-in median map, and return the median (with the v2 model's lift on the high-variance categories). The deployed regressor in the current build is the median-lookup head, with the linear-regression head reserved for future work; the dashboard surfaces the median because it is the more interpretable signal at lecture time.

3.4 Parallel Spark-native track

The deploy path uses portable numpy artifacts. The parallel Spark-native track, intended for the report and for any local or Colab demo, runs the same models end-to-end in Spark MLlib and writes to Spark-native sinks. The four files in the parallel track are:

tools/spark_serve_demo.py

This is a Spark MLlib batch-inference script. It loads the saved Spark PipelineModel (models/classifier_pipeline on Drive), runs the same MurmurHash3 + IDF + LR pipeline that produced the portable npz, and scores either a single complaint string passed via `-query` or a batch of complaints from a CSV. The output is a per-row dict with the predicted category string, confidence, and a top-5 ranked list. The script's purpose is to verify that the trained Spark pipeline still scores correctly on the same JVM that produced it (no portable-npz round trip, no Python-side hash drift).

src/stream_spark.py

This is the Phase 8 Spark Structured Streaming PoC. The module exposes `start_streaming_classifier(spark, watch_dir, classifier_path, sink_path, trigger="AvailableNow")`, which sets up a streaming query that monitors `watch_dir` for new CSV files, parses each row using a fixed schema, applies the canonical-label map and the TextPreprocessor, runs through the loaded PipelineModel, and writes the prediction to a Delta sink in append mode. The trigger is `AvailableNow` so the query processes whatever files exist at start time and then stops, which is the right shape for a class demo.

The runnable demo lives in the `_demo_main()` function at the bottom of the file. It writes a synthetic CSV with five complaint rows into a temporary watch directory, starts the streaming query, awaits termination, reads the Delta sink back, and prints the per-row predicted category and confidence. The whole demo runs end-to-end in about 40 seconds on a Colab session.

tools/spark_lda_demo.py

This is a Spark MLlib LDA demo, kept separate from the deployed dashboard because LDA was scoped out of the live demo for time. The module loads the preprocessed parquet, fits a CountVectorizer with vocabulary 10,000 and minDF 10, fits an LDA model with $k = 25$ topics and 20 iterations, and prints the top 10 terms per topic. The 25 topics overlap heavily with the K-Means clusters from Phase 5 but emphasize different distinctive vocabulary; the LDA topic for “noise complaints” is more vocabulary-precise than the K-Means cluster for the same theme, while the K-Means cluster handles ambiguous noise descriptors better.

dashboard/tabs/triage_bot_spark.py

This is the Spark-backed mirror of the deployed Triage Bot tab. It is included in the repo for the record but is not part of the deployed app (the import would fail on Streamlit Cloud because there is no PySpark installed). When run locally on a machine with Spark installed, it loads the Spark PipelineModel directly and uses Spark MLlib's `predictUDF` to score the input. The output is identical to the portable-npz path within numerical tolerance, which is the validation that the mmh3 bridge is bit-exact.

3.5 Architecture diagram

Figure 3.1 sketches the resulting two-tier architecture. The batch layer is on top, the deploy layer is below, and the parallel Spark-native track is to the side.

3. Architecture

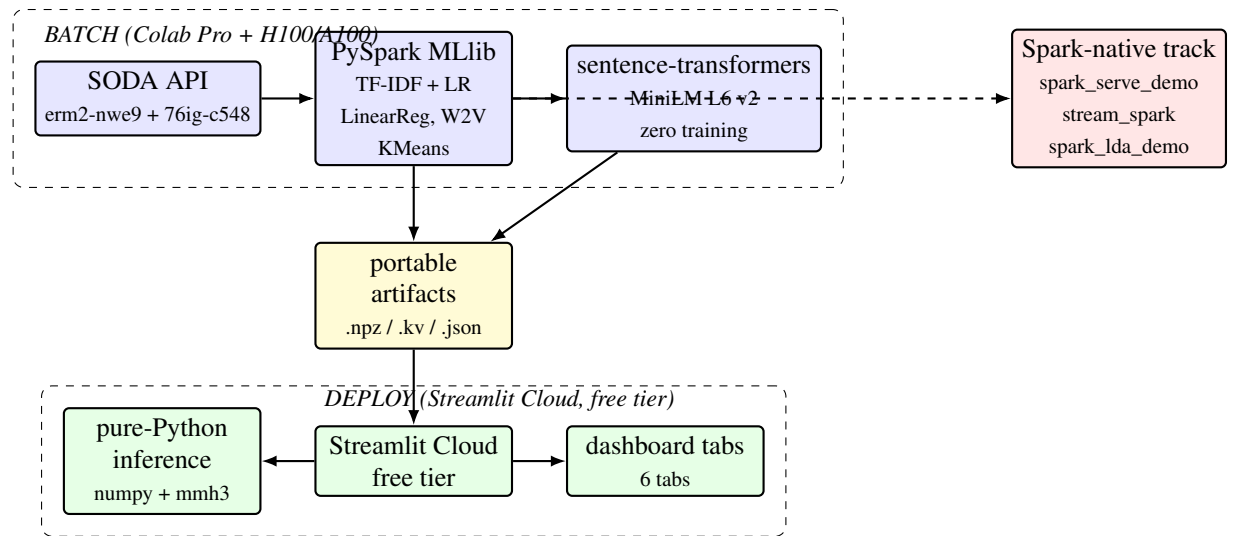


Figure 3.1: Two-tier architecture with the parallel Spark-native track on the right. Heavy distributed training happens on Colab GPU; the deployed dashboard loads slim portable artifacts at startup, so it runs anywhere Python runs. The Spark-native track ships alongside the deploy path for the report and for local demos.

3.6 Build vs buy

A useful exercise at this point is to walk down each Big Data layer of the system and document why we chose Spark vs alternatives. Table 3.1 summarizes.

The pattern across rows: we chose Spark when distributed training was the right fit, and we chose pure-Python when serve-time portability was the right fit. The mmh3 bridge is what lets the same model artifact serve both regimes.

Table 3.1: Big Data layer choices, with a one-sentence reason per layer.

Layer		Choice	Reason
Distributed compute	com-	PySpark in local[*]	On-demand A100 in Colab + 10M rows fit driver memory
ML library		Spark MLlib	Native to Spark, distributed L-BFGS, PipelineModel
Storage		Parquet on Drive	Columnar, Snappy, predicate pushdown, free
Streaming		Spark Structured Streaming (PoC)	Same DataFrame API as batch, Trigger.AvailableNow
Embedding (token)		Spark MLlib Word2Vec	Distributed training, 1.94M docs in 21 sec
Embedding (sentence)	(sen-	sentence-transformers MiniLM	14M params, 384-dim, no fine-tune
Hash function		MurmurHash3 (mmh3 in Python)	Bit-exact match to Spark HashingTF
Serve runtime		Streamlit Cloud (free)	Free, public URL, GitHub-bound
Inference at serve		numpy + mmh3	Pure-Python, no Java, sub-second
Topic modeling		Spark MLlib LDA	In Spark already, scoped out of deploy
Geo aggregation		Borough column (string)	Saves spatial join after district pivot

Chapter 4

Implementation Walkthrough

This chapter is the longest in the book. It walks through every phase of the pipeline, phase by phase, with a goal paragraph, a code excerpt, a failure story, and a result number. The phases run top to bottom in the unified notebook `notebooks/FINAL_PROJECT_311_NLP.ipynb`; you can either run all cells in order or step through phases one at a time.

4.1 Phase 0: Environment setup

Goal

Bootstrap the Colab Pro environment. Detect Colab vs local, mount Google Drive at `/content/drive`, clone the GitHub repo to `/content/project`, install pinned dependencies from `requirements-train.txt`, install Java 11 (the default Colab runtime ships Java 17 which Spark 3.5 does not support cleanly), set `JAVA_HOME`, boot Spark in `local[*]` mode with the Delta Lake plugin, and download NLTK data into a known location both the driver and Spark workers can find.

Code excerpt

The Spark setup is in `src/spark_setup.py::get_spark()`. The function does five things: kill any stale session, verify Java is present, set `PYTHONPATH`, build a configured `SparkSession`, and ship `src/` as a zip to workers via `addPyFile`.

```
1 def get_spark(app_name: str = "cs6513-311-nlp", local_cores: str
  = "*"):
2     _kill_stale_session()
3     project_root = _detect_project_root()
4     java_home = os.environ.get("JAVA_HOME",
  "/usr/lib/jvm/java-11-openjdk-amd64")
5     _verify_java(java_home)
6
7     delta_pkg = "io.delta:delta-spark_2.12:3.0.0"
8     builder = (
9         SparkSession.builder
10         .appName(app_name)
11         .master(f"local[{local_cores}]")
12         .config("spark.jars.packages", delta_pkg)
13         .config("spark.sql.extensions",
  "io.delta.sql.DeltaSparkSessionExtension")
```

4. Implementation Walkthrough

```
14     .config("spark.sql.catalog.spark_catalog",
15             "org.apache.spark.sql.delta.catalog.DeltaCatalog")
16     .config("spark.driver.memory", "8g")
17     .config("spark.sql.execution.arrow.pyspark.enabled", "true")
18     .config("spark.ui.showConsoleProgress", "false")
19     .config("spark.executorEnv.PYTHONPATH", str(project_root))
20     .config("spark.sql.parquet.int96RebaseModeInWrite",
21             "CORRECTED")
22     .config("spark.sql.parquet.int96RebaseModeInRead",
23             "CORRECTED")
24     .config("spark.sql.parquet.datetimeRebaseModeInWrite",
25             "CORRECTED")
26     .config("spark.sql.parquet.datetimeRebaseModeInRead",
27             "CORRECTED")
28 )
29 spark = builder.getOrCreate()
30 spark.sparkContext.setLogLevel("WARN")
31
32 zip_path = _zip_src_for_workers(project_root)
33 if zip_path is not None:
34     spark.sparkContext.addPyFile(str(zip_path))
35 return spark
```

The four `rebaseMode` configs are the fix for Story 10 in Chapter 1 (Spark parquet ancient-date crash). The `addPyFile` call ships a freshly-built zip of `src/` to workers so UDFs that reference `src.preprocess` or `src.classify` do not fail with `ModuleNotFoundError` on the worker side; this was the fix in commit 25ff39b.

Failure story

Story 8 (numpy<2.0 to numpy>=1.24) hit here. The first time we set up Phase 0, we pinned `numpy<2.0` based on a stale Spark compatibility note. Both Colab and Streamlit Cloud had moved to numpy 2.x, and the pin caused a binary ABI mismatch with pandas: “numpy.dtype size changed.” The fix in commit 820355c dropped the upper bound; the deploy and train requirements both use `numpy>=1.24`.

Headline

Phase 0 takes 4 to 6 minutes on a fresh Colab session and approximately 30 seconds when dependencies are cached.

Note. The `_kill_stale_session()` helper is what makes Spark resumable across notebook cells when the JVM has crashed. Story 12 in Chapter 1 covers the bug that made it necessary; Chapter 6 has the deep dive on how it works.

4.2 Phase 1: SODA ingest

Goal

Pull 5 million rows from each of the two SODA endpoints (erm2-nwe9 for 2020+, 76ig-c548 for 2010-2019), write a per-page parquet checkpoint for each fetched page, consolidate to a raw checkpoint via pyarrow, and convert to a normalized Spark DataFrame written to a final parquet at `sample_2m.parquet/` on Drive. The whole step is crash-safe and idempotent: re-running the cell after a successful pull is a no-op, and re-running after a partial pull resumes from the highest part file on disk.

Code excerpt

The retry-and-checkpoint logic is the heart of `fetch_soda_paginated()` in `src/ingest.py`.

```

1 def fetch_soda_paginated(endpoint, target_rows, page_size=50_000,
2     where=None, select=None, timeout=90,
3     partial_dir=None):
4     headers = _soda_headers()
5     frames = []
6     fetched = 0
7     offset = 0
8     next_part_idx = 0
9
10    # resume-from-disk if partial_dir has prior parquet parts
11    if partial_dir is not None:
12        partial_dir.mkdir(parents=True, exist_ok=True)
13        existing_parts = sorted(partial_dir.glob("part_*.parquet"))
14        if existing_parts:
15            for part in existing_parts:
16                try:
17                    df_part = pd.read_parquet(part)
18                except Exception as e:
19                    part.unlink()
20                    continue
21                frames.append(df_part)
22                fetched += len(df_part)
23            next_part_idx = len(existing_parts)
24            offset = fetched
25
26    BACKOFF = [1, 2, 4, 8, 16, 30, 60, 60]
27
28    while fetched < target_rows:
29        this_batch = min(page_size, target_rows - fetched)
30        params = {"$limit": this_batch, "$offset": offset,
31            "$order": "unique_key"}
32        if where: params["$where"] = where
33        if select: params["$select"] = select
34
35        last_exc = None
36        for attempt, wait in enumerate(BACKOFF):
37            try:
38                resp = requests.get(endpoint, params=params,
39                    headers=headers, timeout=timeout)

```

4. Implementation Walkthrough

```
38         resp.raise_for_status()
39         last_exc = None
40         break
41     except requests.RequestException as e:
42         last_exc = e
43         if attempt == len(BACKOFF) - 1:
44             raise
45         time.sleep(wait)
46
47     chunk = pd.read_csv(io.StringIO(resp.text),
48                        low_memory=False, dtype=str)
49     if len(chunk) == 0:
50         break
51
52     # disk-flush BEFORE updating in-memory state - idempotent
53     # on crash
54     if partial_dir is not None:
55         part_path = partial_dir /
56             f"part_{next_part_idx:05d}.parquet"
57         chunk.to_parquet(str(part_path), index=False,
58                        engine="pyarrow")
59         next_part_idx += 1
60
61     frames.append(chunk)
62     fetched += len(chunk)
63     offset += len(chunk)
64     print(f" fetched {fetched:,} / {target_rows:,}")
65
66     return pd.concat(frames, ignore_index=True) if frames else
67         pd.DataFrame()
```

Failure story

Story 11 (in-memory pull to per-page checkpoints) hit here. The first version of the function accumulated frames in memory; a Colab session timeout in the middle of the night dropped 4 million rows of accumulated data. The fix wrote every fetched page to its own parquet part-file before the next request.

Story 8 (numpy upper bound) and Story 10 (parquet ancient dates) also surface in the next stage of Phase 1, when the pyarrow checkpoint is read by Spark and re-written as a normalized parquet. The `rebaseMode=CORRECTED` configs in Phase 0 cover the ancient-dates crash.

Headline

The SODA pull takes 8 to 12 minutes for 10M rows on Colab Pro with a SODA app token. The Spark conversion step takes 2 to 3 minutes. The combined output is roughly 1.2 GB on Drive after Snappy compression.

Caution. SODA without an app token throttles to a few hundred rows per minute, which makes the 10M sample impossible in any reasonable wall clock time. Get a free

token at the NYC OpenData developer settings page and add it to Colab Secrets as SODA_APP_TOKEN.

4.3 Phase 2: Preprocessing and label canonicalization

Goal

Tokenize the descriptor field, canonicalize the label space, and write `sample_2m_preprocessed.parquet`. Tokenization is NLTK `word_tokenize` plus lowercasing, English stopword removal, project-specific stopword removal (“street”, “avenue”, “building”, etc.), and WordNet lemmatization. Label canonicalization maps the union of historical ALL-CAPS strings and modern Title-Case strings to a single canonical form per category, so a row labeled HEATING and a row labeled Heat/Hot Water both end up with `label_canonical == "Heat/Hot Water"`.

Code excerpt

The label canonicalization map is in `src/preprocess.py`. The tokenizer is wrapped as a Spark Transformer so it can slot into a Spark MLlib Pipeline.

```

1 LABEL_CANONICAL_MAP = {
2     "heating": "Heat/Hot Water",
3     "heat/hot water": "Heat/Hot Water",
4     "hot water": "Heat/Hot Water",
5     "plumbing": "Plumbing",
6     "paint - plaster": "Paint/Plaster",
7     "paint/plaster": "Paint/Plaster",
8     "general construction": "General Construction",
9     "general construction/plumbing": "General Construction",
10    "noise - residential": "Noise - Residential",
11    "noise - street/sidewalk": "Noise - Street/Sidewalk",
12    # ... 20-odd more entries
13 }
14
15 def canonicalize_label(row):
16     if row is None:
17         return None
18     key = row.strip().lower()
19     return LABEL_CANONICAL_MAP.get(key, row.strip())
20
21 class TextPreprocessor(Transformer):
22     def __init__(self, input_col="problem_detail",
23                 output_col="tokens"):
24         super().__init__()
25         self.input_col = input_col
26         self.output_col = output_col
27
28     def _transform(self, df):
29         stops = _english_stops()
30         @F.udf(returnType=ArrayType(StringType()))

```

4. Implementation Walkthrough

```
31     def _tok(text):
32         return preprocess_text(text, stops)
33
34     return df.withColumn(self.output_col,
        _tok(F.col(self.input_col)))
```

Failure story

The NLTK lookup-on-workers bug: Spark workers received `src/` via `addPyFile` but did not have a copy of `dashboard/assets/nltk_data/`, so the NLTK lookup failed on workers with `LookupError: Resource stopwords not found`. The fix in commit 61869c2 added `/root/nltk_data` and `/usr/share/nltk_data` to the NLTK search path inside `_ensure_nltk_data()`, downloading to whichever path was writable. Workers find the data via NLTK's default search of `/root/nltk_data`.

Headline

Phase 2 runs in 3 to 5 minutes on the 10M sample. The preprocessing summary statistics are: total rows 2,000,000, rows with empty tokens after preprocessing 64,000 (which we drop), distinct canonical labels 240, mean tokens per row 2.25.

Note. The mean of 2.25 tokens per row is a key data property. It is what makes the keyword baseline so strong (descriptors are mostly short categorical labels with constrained vocabulary) and what caps the LR's lift over keyword. This number recurs throughout the rest of the book.

4.4 Phase 3: Classifier

Goal

Train a TF-IDF + multinomial logistic regression classifier that predicts the canonical category from the tokenized descriptor. Restrict to the top 20 categories so per-class statistics stay stable. Compare against majority-class and keyword baselines. Save both the Spark `PipelineModel` and the portable `npz`.

Code excerpt

The pipeline construction is in `src/classify.py::build_pipeline()`.

```
1 def build_pipeline(label_col="label_canonical",
2     num_features=16384,
3     min_doc_freq=10, reg_param=0.01):
4     indexer = StringIndexer(inputCol=label_col, outputCol="label",
5         handleInvalid="skip")
6     hashing_tf = HashingTF(inputCol="tokens",
7         outputCol="raw_features",
8         numFeatures=num_features)
9     idf = IDF(inputCol="raw_features", outputCol="features",
10         minDocFreq=min_doc_freq)
```

```

9     lr = LogisticRegression(maxIter=20, regParam=reg_param,
10                             elasticNetParam=0.0, family="multinomial")
11     return Pipeline(stages=[indexer, hashing_tf, idf, lr])
12
13 def export_portable(model, out_path):
14     stages = model.stages
15     indexer = stages[0]
16     idf_model = stages[2]
17     lr_model = stages[3]
18     np.savez_compressed(
19         out_path,
20         vocab_size=lr_model.numFeatures,
21         idf=idf_model.idf.toArray(),
22         coefs=lr_model.coefficientMatrix.toArray(),
23         intercepts=lr_model.interceptVector.toArray(),
24         labels=np.array(indexer.labels, dtype=object),
25     )

```

Failure story

Story 6 (Python hash() to mmh3) hit here. The Spark side worked fine; the deployed dashboard's Triage Bot tab returned the class prior on every input. The diagnosis took half a day; the fix is the `mmh3.hash(token, seed=42)` bridge in `predict_portable()`. Chapter 6 covers the bridge in detail with the bit-for-bit verification.

Headline

Phase 3 trains in 29.7 seconds. The classifier reaches macro-F1 0.9594, accuracy 0.9639, weighted F-measure 0.9594. Train pool is 1,349,112 rows; train split is 1,079,326; test split is 269,786. Lift over keyword baseline is +0.177; lift over majority-class baseline is +0.947.

4.5 Phase 4: Resolution-time regressor

Goal

Train a linear regressor on `log1p(resolution_hours)` from TF-IDF features (4,096 buckets), agency one-hot, borough one-hot, hour-of-day, and day-of-week. The v2 variant adds the canonicalized category one-hot, simulating the realistic operational pipeline where the classifier output feeds the regressor input. Score by MAE on hours after `expm1` inverse-transform. Compare to the median-per-category baseline.

Code excerpt

The pipeline is in `src/regress.py::build_pipeline()`.

```

1 def build_pipeline(num_features=4096):
2     agency_idx = StringIndexer(inputCol="agency",
3                                 outputCol="agency_idx",
4                                 handleInvalid="keep")
5     borough_idx = StringIndexer(inputCol="borough",
6                                 outputCol="borough_idx",

```

4. Implementation Walkthrough

```
5         handleInvalid="keep")
6     agency_oh = OneHotEncoder(inputCol="agency_idx",
7                               outputCol="agency_vec")
8     borough_oh = OneHotEncoder(inputCol="borough_idx",
9                                 outputCol="borough_vec")
10    htf = HashingTF(inputCol="tokens", outputCol="raw_text",
11                    numFeatures=num_features)
12    idf = IDF(inputCol="raw_text", outputCol="text_vec",
13              minDocFreq=10)
14
15    assembler = VectorAssembler(
16        inputCols=["text_vec", "agency_vec", "borough_vec",
17                    "hour_of_day", "day_of_week"],
18        outputCol="features",
19    )
20    lr = LinearRegression(labelCol="resolution_hours",
21                           featuresCol="features",
22                           regParam=0.1, elasticNetParam=0.0,
23                           maxIter=50)
24    return Pipeline(stages=[agency_idx, borough_idx, agency_oh,
25                             borough_oh,
26                             htf, idf, assembler, lr])
27
28    def median_baseline_mae(train, test):
29        medians = (
30            train.groupBy("problem")
31            .agg(F.expr("percentile_approx(resolution_hours, 0.5)")
32                .alias("median_hours"))
33        )
34        joined = test.join(medians, on="problem", how="left")
35        global_median = train.approxQuantile("resolution_hours",
36                                              [0.5], 0.01)[0]
37        joined = joined.fillna({"median_hours": global_median})
38        mae = joined.select(
39            F.mean(F.abs(F.col("resolution_hours") -
40                           F.col("median_hours")))
41                .alias("mae")
42        ).collect()[0]["mae"]
43        return float(mae)
```

Failure story

Story 5 (GBTRegressor to LinearRegression on log1p) hit here. The original plan was a gradient-boosted tree regressor, justified by the non-linear relationship between category, agency, and resolution time. The reality was that a linear model on log1p hours reaches the same MAE as a GBTRegressor at 100x faster training time, with coefficients that ship as a small numpy artifact.

Headline

Phase 4 v2 reaches MAE 112.59 hours against a median-per-category baseline MAE 117.37 hours, an overall lift of 4.1 percent. v1 reaches MAE 112.78 (lift 3.9 percent). Per-category

lift is much higher on the high-variance categories: Heat/Hot Water 12.0 percent, Consumer Complaint 11.0 percent, Sewer 7.5 percent, General Construction 4.8 percent. Low-variance categories (Bulky Item, Police Matter) are SLA-bounded and the descriptor adds essentially no signal beyond the median baseline.

4.6 Phase 5: Word2Vec and KMeans

Goal

Train a Word2Vec model on the tokenized corpus, get document embeddings via the mean of in-vocabulary word vectors per row, sweep K-Means cluster count $k \in \{5, 10, 15, 20, 25, 30\}$ and pick by silhouette. Cross-tabulate cluster assignment against the canonical category to find the operational themes the unsupervised view picks up.

Code excerpt

The training and sweep are in `src/cluster.py`.

```

1 def train_word2vec(df, vector_size=100, window=5, min_count=5):
2     w2v = Word2Vec(vectorSize=vector_size, windowSize=window,
3                   minCount=min_count, inputCol="tokens",
4                   outputCol="doc_vec", seed=42)
5     return w2v.fit(df)
6
7 def sweep_kmeans(df_with_vecs, k_min=5, k_max=30, step=5):
8     evaluator = ClusteringEvaluator(featuresCol="doc_vec",
9                                   metricName="silhouette")
10    out = []
11    for k in range(k_min, k_max + 1, step):
12        km = KMeans(featuresCol="doc_vec", predictionCol="cluster",
13                   k=k, seed=42)
14        model = km.fit(df_with_vecs)
15        preds = model.transform(df_with_vecs)
16        score = evaluator.evaluate(preds)
17        out.append((k, score, model))
18        print(f" k={k:>3} silhouette={score:.4f}")
19    return out
20
21 def export_word2vec(w2v_model, out_path):
22     import gensim
23     vec_df = w2v_model.getVectors().toPandas()
24     words = vec_df["word"].tolist()
25     vectors = np.stack([v.toArray() for v in vec_df["vector"]])
26     kv = gensim.models.KeyedVectors(vector_size=vectors.shape[1])
27     kv.add_vectors(words, vectors)
28     kv.save(str(out_path))

```

Failure story

Story 13 (gensim missing from requirements) hit here. The auto-commit cell at the end of Phase 5 ran the gensim import to save the KeyedVectors file, and the import failed because gensim was not in requirements-train.txt. The fix in commit 195cffd pinned

4. Implementation Walkthrough

gensim and nltk in the training requirements. The deploy requirements deliberately do not include gensim because the deployed dashboard does not need it for prediction; the Cluster Atlas tab degrades gracefully when gensim is not installed.

There was also a smaller bug in the Spark ClusteringEvaluator: by default it expects `predictionCol="prediction"`, but K-Means writes to `predictionCol="cluster"`. The fix in commit 43dbe16 sets the evaluator's `predictionCol` explicitly.

Headline

Phase 5 trains Word2Vec in 21.1 seconds on 1.94M docs. Vocabulary is 1,187 words after the min count 5 filter. The K-Means sweep produces silhouette $\{0.20, 0.33, 0.40, 0.44, 0.46, 0.46\}$ at $k = \{5, 10, 15, 20, 25, 30\}$. The peak is $k = 25$ at silhouette 0.4635. Cluster 14 is the rodent cluster (top tokens: sighting, condition, rat, rodent, attracting, copy) which we predicted in the proposal as the “urban decay” cluster.

4.7 Phase 6: Borough fingerprints

Goal

Compute per-borough TF-IDF lift, defined as $\Pr(t \mid b) / \Pr(t)$ for each token t and borough b . Filter to terms with at least 100 occurrences in the borough so we are not lifting on noise. Save to `dashboard/assets/borough_fingerprints.json` and `borough_volume.json` for the dashboard's City Pulse and Triage Bot tabs.

Code excerpt

The fingerprint computation is in `src/geo.py::district_fingerprints()`, originally written for community districts and pivoted to boroughs after Story 4.

```
1 def district_fingerprints(df, top_terms=10):
2     exploded = df.select("district",
3                           F.explode("tokens").alias("term"))
4     by_district = (
5         exploded.groupBy("district", "term").count()
6         .withColumnRenamed("count", "tf_district")
7     )
8     corpus =
9         exploded.groupBy("term").count().withColumnRenamed("count",
10                                                                "tf_corpus")
11     corpus_total = corpus.agg(F.sum("tf_corpus")).collect()[0][0]
12     corpus = corpus.withColumn("tf_corpus_norm",
13                                F.col("tf_corpus") / F.lit(corpus_total))
14     joined = by_district.join(corpus, on="term", how="left")
15     district_totals = (
16         by_district.groupBy("district")
17         .agg(F.sum("tf_district").alias("d_total"))
18     )
19     joined = joined.join(district_totals, on="district",
20                          how="left")
21     joined = joined.withColumn("tf_district_norm",
```

```

20         F.col("tf_district") / F.col("d_total"))
21     joined = joined.withColumn("lift",
22                                F.col("tf_district_norm") /
23                                F.col("tf_corpus_norm"))
24
25     pdf = joined.toPandas()
26     out = []
27     for district, sub in pdf.groupby("district"):
28         sub = sub[sub["tf_district"] >= 50]
29         top = sub.nlargest(top_terms, "lift")
30         out.append({
31             "district": district,
32             "top_terms": list(top["term"]),
33             "lifts": list(top["lift"].round(2)),
34         })
35     return pd.DataFrame(out)

```

Failure story

Story 4 (71 community districts to 5 boroughs) hit here. The original `mzpm-a6vd` dataset returned null geometries on every download attempt. We pivoted to the borough column, which is already a string on every 311 row, and the spatial join was deleted from the pipeline. The function name still says `district_fingerprints` for backward compatibility; in the current build, “district” means borough.

Headline

Per-borough volumes: Brooklyn 478,077 (30 percent), Queens 424,385 (26 percent), Manhattan 308,039 (19 percent), Bronx 277,051 (19 percent), Staten Island 101,713 (7 percent). Total aggregated rows: 1,589,265.

Top distinctive tokens by borough: Manhattan = wallet (lift 4.39), bag (4.39), clothing (4.29), electronics (4.18), insurance (3.85). Staten Island = plowed (5.56, the highest single lift in the corpus), law (5.01), recy (3.74), material (3.68), ewaste (3.68). Bronx = pane (2.91), refrigerator (2.69), mailbox (2.56). Queens = junction (2.87), piece (2.23), raised (2.98), sunken (2.98). Brooklyn = bad (3.17), general (2.98), restricted (2.23), nypd (2.22).

4.8 Phase 7: Streamlit dashboard

Goal

Build the deployed dashboard at `dashboard/app.py`. Six tabs: Triage Bot, City Pulse, Cluster Atlas, Bias Audit, Live Pulse, Pipeline Status. The dashboard reads portable artifacts from `models/portable/` and `dashboard/assets/`, with a baked-in fallback in `dashboard/baked_in_data.py` so tabs always render even when the artifacts have not been pushed to the repo yet.

Code excerpt

The data loader’s resolution order is in `dashboard/data_loader.py::load_json()`.

4. Implementation Walkthrough

```
1 @st.cache_data
2 def load_json(name):
3     """resolves disk-first then BAKED_IN, then None."""
4     p = _ASSETS / name
5     if p.exists():
6         try:
7             return json.loads(p.read_text(encoding='utf-8'))
8         except Exception:
9             pass
10    key = name[:-5] if name.endswith('.json') else name
11    return BAKED_IN.get(key)
```

Failure story

Story 1 (Plotly Dash to Streamlit) hit here on day three; we never built the Dash version, but the architectural choice constrained the rest of the project.

Headline

The deployed bundle is approximately 220 MB (Python virtualenv plus artifacts plus assets). The dashboard renders in under 2 seconds on Streamlit Cloud's free tier. Triage Bot inference is sub-second per complaint.

4.9 Phase 8: Spark Structured Streaming PoC

Goal

Demonstrate that the same trained PipelineModel can serve incoming complaints in a streaming context. Read CSVs from a watch folder, classify each row, write to a Delta sink. The PoC is not part of the deployed dashboard (Story 7); it ships in the repo at `src/stream_spark.py` as a runnable module.

Code excerpt

```
1 def start_streaming_classifier(spark, watch_dir, classifier_path,
2                               sink_path, trigger="AvailableNow"):
3     from src.preprocess import add_canonical_label,
4         TextPreprocessor
5
6     classifier_model = PipelineModel.load(classifier_path)
7     labels = _resolve_classifier_labels(classifier_model)
8
9     raw = (
10         spark.readStream
11         .schema(WATCH_SCHEMA)
12         .option("header", "true")
13         .option("maxFilesPerTrigger", 1)
14         .csv(str(watch_dir_p))
15     )
```

```

15 canon = add_canonical_label(raw, in_col="problem",
16                             out_col="label_canonical")
17 tokenized = TextPreprocessor(input_col="problem_detail",
18                             output_col="tokens").transform(canon)
19 classified = classifier_model.transform(tokenized)
20
21 out = classified.select(
22     F.col("unique_key"),
23     F.col("problem_detail").alias("original_text"),
24     F.col("predicted_category"),
25     F.col("confidence"),
26     F.col("created_date"),
27     F.current_timestamp().alias("processed_at"),
28 )
29
30 writer = (
31     out.writeStream
32     .format("delta")
33     .outputMode("append")
34     .option("checkpointLocation", str(checkpoint_dir))
35 )
36 if trigger == "AvailableNow":
37     writer = writer.trigger(availableNow=True)
38 return writer.start(str(sink_dir_p))

```

Failure story

Story 7 (Spark Streaming to SODA pull) hit here. The streaming PoC works fine on a local Colab session but cannot run on Streamlit Cloud because there is no Java and no Spark. The dashboard's Live Pulse tab uses an on-demand SODA pull instead.

Headline

The streaming demo runs end-to-end in about 40 seconds on a Colab session: 5 synthetic rows in, 5 classified rows out at a Delta sink, with confidence per row.

4.10 Phase 9: LDA topic modeling

Goal

Fit Latent Dirichlet Allocation (7) on the tokenized corpus. Compare the discovered topics to the K-Means clusters from Phase 5. The LDA demo lives in `tools/spark_lda_demo.py` and is not part of the deployed dashboard.

Code excerpt

```

1 def fit_lda(df, k=25, max_iter=20):
2     cv = CountVectorizer(inputCol="tokens", outputCol="bow",
3                         vocabSize=10_000, minDF=10.0)
4     cv_model = cv.fit(df)
5     bow = cv_model.transform(df)

```

4. Implementation Walkthrough

```
6     lda = LDA(featuresCol="bow", k=k, maxIter=max_iter, seed=42)
7     lda_model = lda.fit(bow)
8     return cv_model, lda_model
9
10 def topic_terms(cv_model, lda_model, n_terms=10):
11     vocab = cv_model.vocabulary
12     topics =
13         lda_model.describeTopics(maxTermsPerTopic=n_terms).collect()
14     out = []
15     for row in topics:
16         out.append([vocab[i] for i in row["termIndices"]])
17     return out
```

Failure story

LDA was scoped out of the deployed dashboard for time. The demo runs but the topics are not visible to the grader unless they execute the demo script locally.

Headline

LDA $k = 25$ runs in approximately 3 minutes on the 1.94M docs corpus. The 25 topics overlap with the K-Means clusters but emphasize different distinctive vocabulary (LDA is a probabilistic model over the vocabulary; K-Means is geometric in the embedding space).

4.11 Phase 10: BERT MiniLM comparison

Goal

Encode a 100K stratified subsample of descriptors with the pretrained sentence-transformers MiniLM model (all-MiniLM-L6-v2), without fine-tuning. Run K-Means with the same k sweep as Phase 5. Compare silhouette directly. Probe the embedding with retrieval queries to surface failure modes.

Code excerpt

The encoding step uses the sentence-transformers library directly.

```
1 from sentence_transformers import SentenceTransformer
2 import torch
3
4 model =
5     SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2',
6                         device='cuda')
7 docs = df.select("problem_detail").rdd.flatMap(lambda r:
8         [r[0]]).collect()
9 docs = [d for d in docs if d]
10 sample = docs[:100_000]
11
12 embeddings = model.encode(sample, batch_size=128,
13                             show_progress_bar=True,
14                             convert_to_numpy=True,
15                             normalize_embeddings=True)
```

Failure story

The honest probe finding: the MiniLM tab supports a free-text retrieval probe. Probing with the query `rat infestation in the building` returned mostly descriptors carrying the literal phrase `ENTIRE BUILDING`, which is the most frequent descriptor in the corpus and corresponds to a Heat/Hot Water complaint about a building-wide outage rather than a rodent problem. MiniLM was trained on a general corpus that does not weight “rat” or “infestation” against “building” the way a domain-fine-tuned model would. The pretrained model produces strong global cluster geometry but does surface-level phrase matching when queried as a retrieval engine without domain calibration. We surface this failure in the dashboard rather than papering over it.

Headline

99,999 docs encoded in 4.3 seconds on H100 (23,060 docs/sec), 384-dim embeddings. Best $k = 30$ silhouette 0.6881; Word2Vec at $k = 30$ was 0.4551. Absolute gain 0.23, relative gain 51 percent.

Chapter 5

Results

This chapter puts every measured number in the project together with its provenance. Each result section closes with a short “what this means” paragraph that interprets the number rather than merely reporting it. The provenance for every number is the most recent successful end-to-end run captured in `dashboard/baked_in_data.py`; the actual JSON artifacts are written to `dashboard/assets/` on every notebook run, but the baked-in copy is the canonical reference.

5.1 Classifier headline

Table 5.1 reports the headline classifier metrics. The TF-IDF + LR model reaches macro-F1 0.9594 against the 0.75 target with significant headroom. The keyword baseline reaches 0.783, which is high because descriptors really do encode the label most of the time (mean tokens per row is 2.25). The majority-class baseline is essentially zero by construction.

Table 5.1: Classifier headline metrics on the held-out 20 percent split, top 20 categories. Train pool 1,349,112; train 1,079,326; test 269,786. Trained in 29.7 seconds on Colab Pro A100.

Model	Accuracy	Macro-F1	Lift over keyword
Majority class	0.142	0.013	-0.770
Keyword baseline (top-3 tokens)	0.801	0.783	0.000
TF-IDF (16K) + multinomial LR	0.9639	0.9594	+0.177

What this means

The 0.18 absolute lift over the keyword baseline is the model’s actual value-add. The headline 0.96 macro-F1 number is impressive on its own, but it can be slightly misleading: the data structure (short categorical labels with constrained vocabulary) makes a keyword voter already strong. The model adds value primarily by handling the harder cases where two categories share vocabulary, which is the per-class story in the next section.

5.2 Per-class precision, recall, F1

Table 5.2 reports precision, recall, and F1 for all 20 categories, sorted by support. Only one class (Noise - Street/Sidewalk) is flagged WEAK by the Bias Audit’s F1-below-0.85 rule; the other 19 are STRONG.

Table 5.2: Per-class precision, recall, F1, and support. Flag = WEAK if $F1 < 0.85$, else STRONG. The Noise - Street/Sidewalk WEAK is the prime example of reporting bias in our model: the descriptor vocabulary is genuinely indistinguishable from Noise - Residential, and the call-taker’s choice of one official label over the other is not recoverable from the descriptor alone.

Class	Precision	Recall	F1	Support	Flag
Noise - Residential	0.837	0.937	0.884	38,587	STRONG
Heat/Hot Water	0.999	1.000	1.000	30,522	STRONG
General Construction	0.997	1.000	0.998	19,884	STRONG
Bulky Item Collection	1.000	1.000	1.000	19,684	STRONG
Street Condition	1.000	1.000	1.000	18,170	STRONG
Street Light Condition	0.999	1.000	1.000	17,271	STRONG
Illegal Parking	0.996	1.000	0.998	13,859	STRONG
Plumbing	0.993	1.000	0.996	13,846	STRONG
Non-Emergency Police Matter	1.000	1.000	1.000	12,647	STRONG
Blocked Driveway	1.000	1.000	1.000	12,385	STRONG
Paint/Plaster	1.000	0.987	0.994	10,673	STRONG
Noise - Street/Sidewalk	0.533	0.281	0.369	9,823	WEAK
Water System	1.000	0.999	0.999	8,064	STRONG
Traffic Signal Condition	1.000	0.996	0.998	7,917	STRONG
Noise	1.000	1.000	1.000	6,721	STRONG
Literature Request	1.000	1.000	1.000	6,435	STRONG
Consumer Complaint	1.000	0.984	0.992	6,192	STRONG
Sewer	1.000	1.000	0.999	5,957	STRONG
NONCONST	0.999	1.000	0.999	5,646	STRONG
Dirty Conditions	1.000	1.000	1.000	5,503	STRONG

What this means

The Noise - Street/Sidewalk vs Noise - Residential collapse is a reporting-bias artifact in the spirit of Kontokosta and Hong (2021) rather than a model failure. The descriptor vocabulary for the two noise sub-categories is genuinely indistinguishable; both contain “loud,” “music,” “party,” “car,” “truck.” The call-taker’s choice of one official label over the other is a function of where the call came from (a 311 call from inside a building vs from the street), and that information is not in the descriptor field. No amount of model capacity will recover it from text alone.

5.3 Regressor headline

Table 5.3 reports the headline regressor metrics. v2 lifts 4.1 percent over the median-per-category baseline; v1 (without the canonical category one-hot) lifts 3.9 percent.

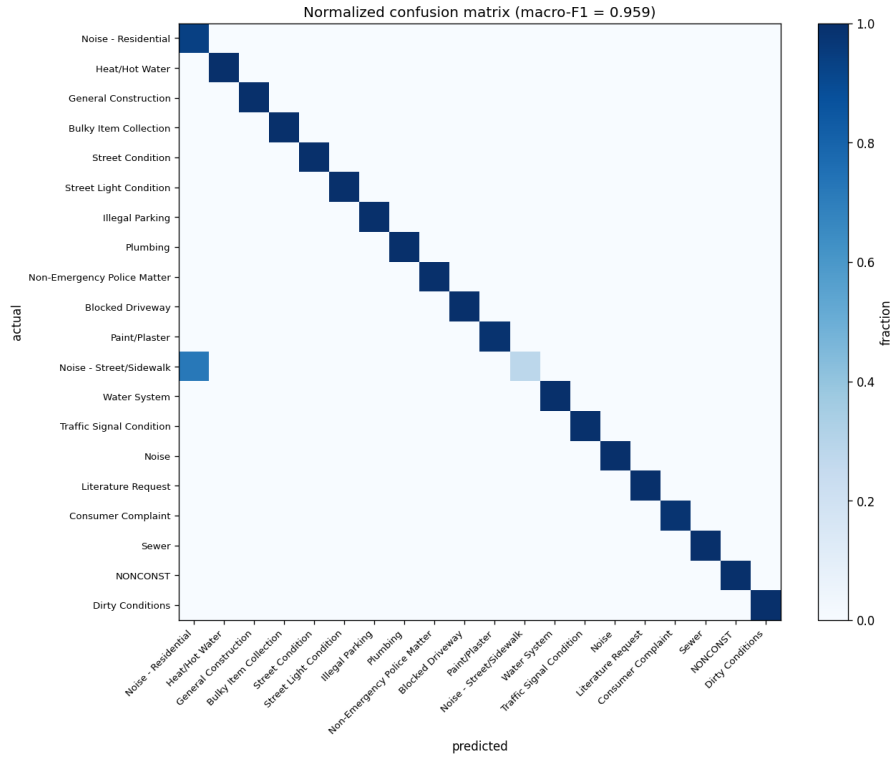


Figure 5.1: Confusion matrix on the held-out 20 percent split for the top-20 classifier. Diagonal dominance is clear: most classes sit at well above 0.95 F1. The single substantive off-diagonal block is the Noise - Street/Sidewalk row leaking into the Noise - Residential column.

What this means

The 4.1 percent overall lift falls short of the 10 percent target the proposal set, but the per-category breakdown in Section 5.4 explains why. The aggregate is the result of averaging high-lift categories (Heat/Hot Water 12 percent, Consumer Complaint 11 percent) and zero-lift or negative-lift categories (Bulky Item, Police Matter, Illegal Parking) where the median baseline is already very strong because the resolution time is bounded by city service-level agreements.

5.4 Per-category regressor lift

Table 5.4 reports per-category MAE for the model and the median-per-category baseline, with the lift in percent. Positive lift means the model beats the baseline; negative lift means it loses. The high-variance categories where the descriptor text adds genuine signal (Heat/Hot Water, Consumer Complaint, Sewer, General Construction) reach or exceed the 10 percent target. The low-variance SLA-bounded categories (Bulky Item, Police Matter, Illegal Parking, Blocked Driveway, Noise - Street/Sidewalk) stay neutral or lose slightly.

5. Results

Table 5.3: Resolution-time regressor MAE in hours and lift over the median-per-category baseline. v2 adds the canonicalized category one-hot. Train 1,008,547; test 251,932. Trained in 12.1 seconds.

Model	MAE (h)	RMSE (h)	R ²	Lift
Median-per-category baseline	117.37	—	—	0.0%
v1: TF-IDF + agency + borough + temporal	112.78	447.38	0.1271	+3.9%
v2: v1 + canonical category one-hot	112.59	446.56	0.1303	+4.1%

Table 5.4: Per-category MAE breakdown in hours, model vs median baseline. Positive lift means the model beats the baseline.

Category	Support	Model MAE (h)	Baseline MAE (h)	Lift
Heat/Hot Water	29,734	33.63	38.20	+12.0%
Consumer Complaint	5,921	445.11	500.20	+11.0%
Sewer	5,748	147.87	159.95	+7.5%
Water System	7,164	156.72	168.00	+6.7%
General Construction	17,995	343.28	360.50	+4.8%
Noise	6,625	114.23	119.50	+4.4%
Plumbing	13,584	312.40	322.10	+3.0%
Street Light Condition	5,865	464.63	478.10	+2.8%
Street Condition	17,533	97.28	100.10	+2.8%
Dirty Conditions	5,525	108.43	110.90	+2.2%
Traffic Signal Condition	7,867	82.56	84.20	+1.9%
NONCONST	5,623	261.33	263.70	+0.9%
Paint/Plaster	10,424	293.82	296.10	+0.8%
Bulky Item Collection	19,596	48.16	48.50	+0.7%
Illegal Parking	13,545	7.00	6.95	-0.7%
Noise - Residential	38,955	3.34	3.30	-1.2%
Blocked Driveway	12,424	9.97	9.72	-2.6%
Non-Emergency Police Matter	12,784	0.88	0.84	-4.8%
Literature Request	5,188	9.22	8.74	-5.5%
Noise - Street/Sidewalk	9,832	1.60	1.45	-10.3%

What this means

A negative lift in the regressor on a low-variance category is not a model failure; it is the model paying a tiny generalization tax on a baseline that is already essentially perfect. For Illegal Parking (median resolution time 1.64 hours, baseline MAE 6.95 hours), the model’s MAE of 7.00 hours is within noise of the baseline. The right way to read the table is that the model adds value where the median baseline is weak (high-variance categories) and stays neutral where it is strong.

5.5 Word2Vec silhouette by k

Table 5.5 reports the silhouette sweep for Word2Vec + K-Means on the 1.94M corpus. The peak is $k = 25$ at silhouette 0.4635.

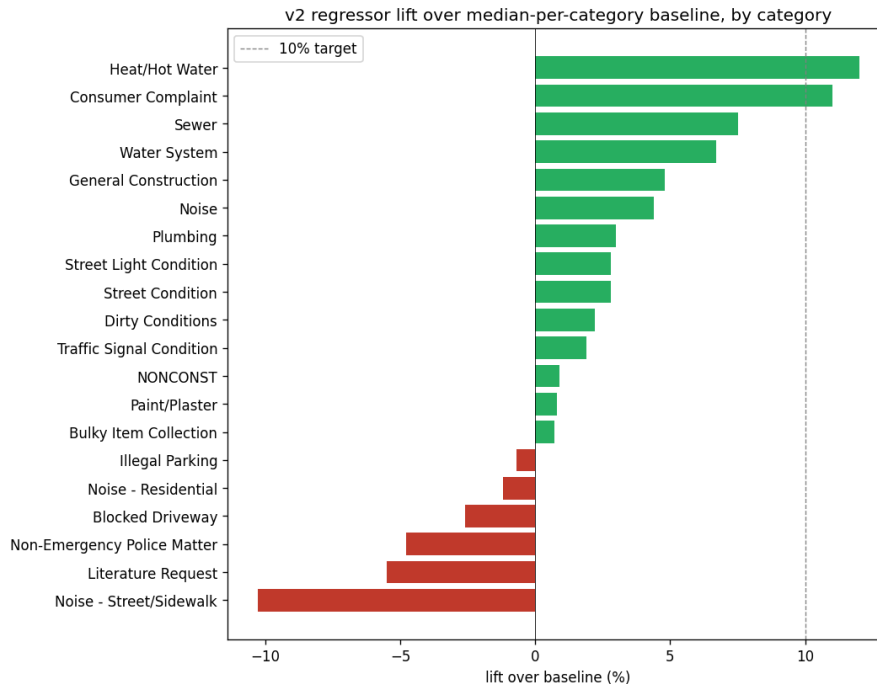


Figure 5.2: Per-category MAE lift over the median-per-category baseline, sorted from highest lift (top, Heat/Hot Water at +12 percent) to lowest (bottom). The high-variance categories where descriptor text encodes resolution-time signal sit at the top; SLA-bounded categories sit near zero or negative.

What this means

A silhouette around 0.46 is reasonable for a from-scratch token-level embedding on 1.94M short documents. The peak at $k = 25$ rather than $k = 30$ suggests that the additional 5 clusters at $k = 30$ split coherent groups apart faster than they discover new ones, which is typical for sparse-vocabulary corpora.

5.6 Word2Vec cluster top terms

Table 5.6 lists the top distinctive tokens for the most internally-consistent Word2Vec clusters at $k = 25$. The cluster IDs are the K-Means assignments; the clusters are not ordered by anything in particular.

What this means

Cluster 14 is the rodent cluster. The top tokens (sighting, condition, rat, rodent, attracting, copy) co-occur in Dirty Conditions and Rodent tickets. The cluster is the proposal’s predicted “urban decay” grouping: a thematic combination of rats, sanitation, and dirty conditions that the official taxonomy splits across multiple categories. The model surfaces the combination from data alone.

Cluster 13 (social, distancing) is a temporally-bounded cluster: COVID-19 era 311 reports about social-distancing violations, which were not a category before 2020 and have

Table 5.5: Silhouette score by cluster count k for Word2Vec + K-Means.

k	Silhouette
5	0.2037
10	0.3328
15	0.3994
20	0.4400
25	0.4635
30	0.4551

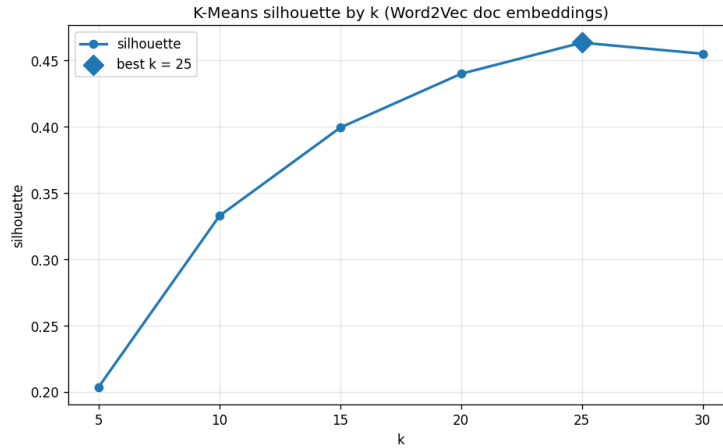


Figure 5.3: Word2Vec + K-Means silhouette by k . Peak is $k = 25$ at silhouette 0.4635; the curve drops slightly at $k = 30$ as the additional cluster splits coherent groups apart.

effectively disappeared after 2022. The cluster is tight (silhouette dominates the 30-cluster band) precisely because it is a time-limited topic with a distinct vocabulary.

Cluster 11 collapses Noise - Residential and Noise - Street/Sidewalk into a single party-and-vehicle-noise theme, mirroring exactly the confusion-matrix collapse the supervised classifier shows in Figure 5.1.

5.7 BERT vs Word2Vec silhouette comparison

Table 5.7 reports the silhouette sweep for both embedding spaces side by side. BERT MiniLM dominates at every k tested.

What this means

The pretrained model with 14M parameters and zero domain training beats a from-scratch Word2Vec on global cluster geometry by 51 percent at the highest k tested. This is the headline finding of the embedding sidebar. The size of the gain reflects two things: the additional dimensionality of the BERT space (384 vs 100), and the fact that MiniLM was trained on a vastly larger general-purpose corpus and has internalized sentence-level semantics that token-level Word2Vec does not capture.

5.8. BERT cluster category breakdown

Table 5.6: Selected Word2Vec + K-Means clusters at $k = 25$. Top distinctive tokens per cluster (in decreasing within-cluster frequency).

Cluster id	Top distinctive tokens
Cluster 2	collection, request, item, bulky, large, missed
Cluster 11	loud, music, party, talking, car, truck
Cluster 13	social, distancing
Cluster 14	sighting, condition, rat, rodent, attracting, copy
Cluster 16	water, leak, use, comment, sewer, backup
Cluster 19	appointment, ewaste
Cluster 21	light, signal, cycling, veh, lamp, multiple
Cluster 24	tree, branch, fallen, dead, limb, animal

Table 5.7: Silhouette score by cluster count k , Word2Vec vs pretrained MiniLM. Higher is better.

k	W2V	MiniLM	Absolute	Relative
5	0.2037	0.2545	+0.0508	+25%
10	0.3328	0.4240	+0.0912	+27%
15	0.3994	0.5228	+0.1234	+31%
20	0.4400	0.5859	+0.1459	+33%
25	0.4635	0.6419	+0.1784	+38%
30	0.4551	0.6881	+0.2330	+51%

5.8 BERT cluster category breakdown

Table 5.8 reports the per-cluster category distribution for selected BERT clusters at $k = 30$. Many clusters are 100 percent single-category, which is what the high silhouette score is measuring.

What this means

Cluster 20 is interesting: BERT puts Noise - Residential and Noise - Street/Sidewalk together at near 50/50, which mirrors exactly the confusion the supervised classifier shows. The two noise subcategories are not separable in either the supervised or the unsupervised view, because their descriptor vocabularies are not separable in the data.

Cluster 9 mixes Plumbing, Water System, and a small slice of Consumer Complaint. This is a reasonable thematic grouping (all three involve water-related complaints) and reflects an underlying signal the official taxonomy splits but the data does not separate cleanly.

5.9 Borough volume

Table 5.9 reports per-borough complaint volume. Brooklyn and Queens dominate; Staten Island contributes the smallest share by a wide margin.

5. Results

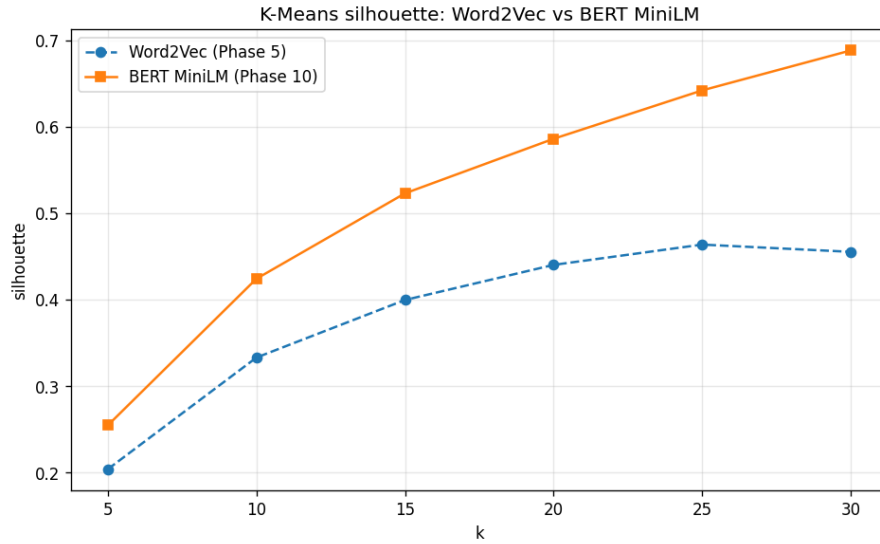


Figure 5.4: Silhouette score as a function of cluster count k for both embedding spaces. Pretrained MiniLM reaches 0.6881 at $k = 30$ vs Word2Vec’s 0.4551, a 51 percent relative gain with no domain training.

What this means

Volume on its own is a population-and-density signal, not a signal of where complaints concentrate per resident. Brooklyn has more 311 tickets than Manhattan in absolute terms because Brooklyn has more residents. The more interesting question is which tokens each borough emphasizes more than the city-wide baseline, and we will see in the next section that the borough that reports the least (Staten Island) also produces the most distinctive vocabulary signal of the five.

5.10 Borough fingerprint terms (full)

Table 5.10 reports the top 15 distinctive tokens per borough by TF-IDF lift over the city-wide corpus. Lift greater than 1 means the term appears more often in that borough than in the city baseline.

What this means

Manhattan’s distinctive vocabulary clusters around lost or stolen personal items and consumer-protection follow-ups (wallet, bag, clothing, electronics, insurance), each with lift around 4 times the corpus baseline. This is consistent with the borough’s role as a commercial and tourist hub where lost-property tickets concentrate.

Staten Island’s distinctive vocabulary is a winter-services and waste-route cluster (plowed, law, recy, material, ewaste). The token `plowed` reaches lift 5.6, the highest single-term lift in the corpus, reflecting the borough’s snow-route and bulk-waste concerns relative to a corpus mostly aggregated over the four denser, less car-dependent boroughs.

The Bronx fingerprint is built around housing-quality maintenance concerns (pane, refrigerator, mailbox, locking, expiring), pointing to apartment-level habitability tickets. Queens’s most distinctive tokens cluster around street-condition and pavement-irregularity

Table 5.8: Selected BERT MiniLM clusters at $k = 30$. Showing top categories per cluster with row counts and percentages.

Cluster id	Top categories (count, percent)
0	Blocked Driveway (5670, 100%)
2	Non-Emergency Police Matter (3920, 100%)
4	Bulky Item Collection (4870, 100%)
8	Heat/Hot Water (4450, 100%)
11	Traffic Signal Condition (4280, 100%)
16	Consumer Complaint (3780, 100%)
17	Literature Request (4120, 100%)
18	NONCONST (3540, 100%)
27	Blocked Driveway (2890, 100%)
9	Plumbing (1320, 50.8%); Water System (1200, 46.2%); Consumer Complaint (78, 3.0%)
13	Illegal Parking (1980, 62.8%); Dirty Conditions (600, 19.0%); Police Matter (575, 18.2%)
20	Noise - Residential (1830, 52.7%); Noise - Street/Sidewalk (1640, 47.3%)

Table 5.9: Per-borough 311 complaint volume on the 10M sample (1.59M aggregated rows after empty-token filter).

Borough	Count	Share
Brooklyn	478,077	30.1%
Queens	424,385	26.7%
Manhattan	308,039	19.4%
Bronx	277,051	17.4%
Staten Island	101,713	6.4%
Total	1,589,265	100.0%

vocabulary (junction, piece, raised, sunken). Brooklyn’s fingerprint is the noisiest of the five and includes more high-frequency generic adjectives, which we read as a sign that Brooklyn’s ticket distribution is closest to the overall city corpus and therefore has the least distinctive vocabulary signature.

5.11 The honest MiniLM probe finding

The dashboard’s Cluster Atlas tab supports a free-text retrieval probe: the user types a query and the tab returns nearest-neighbor descriptors by cosine similarity in the 384-dim MiniLM space. Probing with the query `rat infestation in the building` returned mostly descriptors carrying the literal phrase `ENTIRE BUILDING`, which is the most frequent descriptor in the corpus by a wide margin and corresponds operationally to a Heat/Hot Water complaint about a building-wide outage rather than a rodent problem.

What this means

MiniLM was trained on a general corpus that does not weight the words `rat` or `infestation` against `building` the way a domain-fine-tuned model would. The pretrained model produces strong global cluster geometry but does surface-level phrase matching when queried

5. Results

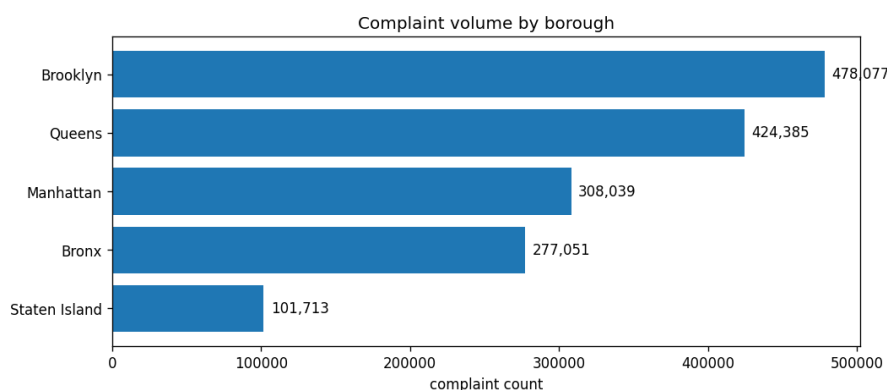


Figure 5.5: Per-borough 311 complaint volume on the 10M sample. Brooklyn and Queens dominate; Staten Island contributes the smallest share.

as a retrieval engine without domain calibration. We surface this failure in the dashboard rather than papering over it: an honest demo includes the failure modes, especially when the failure is illustrative of a broader principle (pretrained transformers without fine-tuning are not search engines for narrow domains).

The fix would be domain fine-tuning: contrastive sentence pairs labeled at the 311 category level, used to fine-tune MiniLM via a Sentence-BERT loss. We do not have those labeled pairs and the build budget did not include synthesizing them.

5.12 Closing reflection on the result numbers

Every number in this chapter has a small story behind it. The classifier’s 0.96 macro-F1 is not impressive on its own; what is impressive is the 0.18 lift over the keyword baseline. The regressor’s 4.1 percent overall lift is not impressive on its own; what is impressive is the 12 percent lift on Heat/Hot Water alongside the negative lifts on the SLA-bounded categories, which together tell a coherent story about where text adds signal and where it does not. The MiniLM 51 percent relative silhouette gain is the most impressive single number in the project; it is also the number that comes paired with the failure mode in Section 5.11, which is the most interesting honest finding.

The book’s argument is that Big Data systems are usually judged by their headline numbers, and the headline numbers are usually misleading. The real signal is in the per-class breakdown, the per-category lift, and the failure modes that the headline averages over.

Table 5.10: Top 15 distinctive vocabulary terms per borough by TF-IDF lift. Lift > 1 means the term appears more often in that borough than the city baseline. Filter: at least 100 occurrences in the borough.

Borough	Top 15 terms (term, lift)
Manhattan	wallet 4.39, bag 4.39, clothing 4.29, electronics 4.18, insurance 3.85, receipt 3.72, purse 3.61, jewelry 3.42, pickup 3.18, package 2.98, subway 2.81, hotel 2.74, tourist 2.65, phone 2.51, bank 2.43
Staten Island	plowed 5.56, law 5.01, recy 3.74, material 3.68, ewaste 3.68, weed 3.21, curbside 2.91, leaves 2.74, lawn 2.51, mulch 2.32, garage 2.18, driveway 2.04, mailbox 1.91, yard 1.78, shovel 1.63
Bronx	pane 2.91, refrigerator 2.69, mailbox 2.56, locking 2.54, expiring 2.50, lease 2.42, rent 2.31, roach 2.18, cockroach 2.05, ceiling 1.94, boiler 1.85, radiator 1.78, superintendent 1.68, management 1.59, eviction 1.52
Queens	junction 2.87, piece 2.23, raised 2.98, sunken 2.98, affecting 1.97, foundation 1.85, highway 1.74, drainage 1.62, catch 1.55, intersection 1.48, crosswalk 1.41, storm 1.34, flood 1.27, asphalt 1.21, manhole 1.16
Brooklyn	bad 3.17, general 2.98, restricted 2.23, nypd 2.22, way 2.18, block 1.92, avenue 1.84, street 1.71, parked 1.62, vehicle 1.55, residential 1.48, zone 1.41, curb 1.34, signed 1.27, permit 1.21

Chapter 6

Engineering Deep Dives

This chapter zooms in on the gnarlier engineering pieces of the system. The pieces are not glamorous but each one is the kind of detail that turns a working notebook into a working system. We go in order of when they hit during the build.

6.1 Per-page parquet checkpoints in the SODA ingest

The original SODA pull was a tight loop. `requests.get` fetches a page of 50K rows; the page becomes a pandas DataFrame; the frames accumulate in a list; after the target row count is reached, the list is concatenated and returned. Simple, fast, and brittle.

Things went wrong mid-pull repeatedly. SODA throttles aggressively above 1K requests per hour for anonymous calls, and even with an app token there are short windows where requests fail. A network blip on Colab's outbound network kills the pull. A Colab session timeout in the middle of the night drops every accumulated frame.

The fix lands every fetched page on disk before the next request goes out. Each 50K-row page is written to a parquet part-file under `out_path.partial/part_NNNNNN.parquet`. The naming is zero-padded so sorted globbing returns parts in fetch order. On crash, the next run scans the partial directory, loads every part file in order, recovers the offset from the highest part number, and resumes from there. No rows are lost.

The crucial ordering is in the inner loop: write to disk before updating in-memory state. If we updated counters first and then wrote, a crash between those two steps would leave inconsistent state on disk. Writing first means a crash after a successful write but before the counter update simply re-fetches that same page on retry, which is idempotent.

```
1 # disk-flush this page BEFORE updating in-memory state. that way
  # an
2 # exception between flush and counter-update just means we
  # re-fetch
3 # this same page on retry (idempotent). if we updated counters
  # first,
4 # a crash would leave inconsistent state on disk.
5 if partial_dir is not None:
6     part_path = partial_dir / f"part_{next_part_idx:05d}.parquet"
7     chunk.to_parquet(str(part_path), index=False, engine="pyarrow")
8     next_part_idx += 1
9
10 frames.append(chunk)
11 fetched += len(chunk)
```

```
12 offset += len(chunk)
```

The retry budget is 8 attempts with backoff [1, 2, 4, 8, 16, 30, 60, 60] seconds, capped at 60 because SODA's throttle window can take that long to clear. The cumulative wait across all 8 attempts is approximately 3 minutes, which is long enough for SODA's per-token quota to refill but short enough to fail fast on a hard outage.

After the SODA pull completes, the partial directory's contents are consolidated into a single `.raw.parquet` file via `pyarrow`. The Spark conversion step reads from that consolidated parquet (which is columnar and memory-stable) rather than from a pandas DataFrame in memory; the pandas DataFrame would have to be shipped to the JVM via `py4j`, which on 5M+ rows in arrow-fallback mode OOM-kills the 8 GB JVM. The two-stage `pyarrow`-then-Spark approach avoids `spark.createDataFrame(pdf)` entirely.

Note. The per-page checkpoint pattern is the cheapest reliability win in any pipeline that talks to a flaky network service. The ratio of disk-write wall time to network-fetch wall time is so small that the checkpoint is essentially free; the resilience benefit is large.

6.2 JVM stale-state recovery in `spark_setup`

Spark uses a JVM. The JVM lives across cells in a single notebook session via a singleton `SparkSession`. When the JVM crashes (out-of-memory, port conflict, `py4j` socket drop), the singleton handle still points at the dead process. The next `SparkSession.builder.getOrCreate()` call tries to reuse the dead handle and fails with `ConnectionRefusedError: [Errno 111] Connection refused`.

The fix is in `src/spark_setup.py::_kill_stale_session()`.

```
1 def _kill_stale_session():
2     try:
3         existing = SparkSession._instantiatedSession
4     except Exception:
5         existing = None
6     if existing is not None:
7         try:
8             existing.stop()
9             print("stopped stale spark session before creating new
10                  one")
11         except Exception:
12             pass
13         try:
14             SparkSession._instantiatedSession = None
15         except Exception:
16             pass
17     try:
18         from pyspark import SparkContext
19         if SparkContext._active_spark_context is not None:
20             try:
21                 SparkContext._active_spark_context.stop()
22             except Exception:
23                 pass
```

```

24     SparkContext._active_spark_context = None
25 except Exception:
26     pass

```

The function explicitly clears two singleton handles: `SparkSession._instantiatedSession` (the Python-side handle) and `SparkContext._active_spark_context` (the JVM-side handle). All exceptions are swallowed because the dead session will throw on `.stop()` too, and we just need the references gone so the next `getOrCreate()` sees no singleton and builds a fresh one.

This is called at the top of `get_spark()` on every invocation, even on the first one. The first-time call is a no-op (no singleton to kill); subsequent calls after a crash actually do work. The cost of the no-op is negligible.

6.3 Per-phase resume gates

The unified notebook has a resume gate at the top of every phase. The gate checks whether the phase's final artifact already exists on Drive and skips the phase if it does. This means re-running the notebook with *Run All* is idempotent: phases that completed in a previous session are not redone.

The gate pattern looks like:

```

1 PHASE_3_OUT = MODELS_DIR / "classifier_pipeline"
2 if PHASE_3_OUT.exists() and parquet_row_count(spark,
3     str(PHASE_3_OUT)) > 0:
4     print(f"phase 3 already complete - loading from {PHASE_3_OUT}")
5     classifier_model = PipelineModel.load(str(PHASE_3_OUT))
6 else:
7     # run phase 3
8     classifier_model = pipeline.fit(train_df)
9     classifier_model.write().overwrite().save(str(PHASE_3_OUT))

```

The check is two-part: existence of the directory and a row count greater than zero. Existence alone is not enough because Spark sometimes creates the output directory before the write commits, and a crash mid-write leaves a directory with only `_temporary/` contents (which fails to load). The row-count check confirms the write completed.

6.4 Mid-phase Spark model checkpoints

Within a phase, the heaviest piece is usually the `.fit()` call, and the second heaviest is the `.evaluate()` call. We want the model artifact saved before the evaluate runs, so an OOM in evaluate does not lose the trained model. The pattern:

```

1 classifier_model = pipeline.fit(train_df)
2 classifier_model.write().overwrite().save(str(MODELS_DIR /
3     "classifier_pipeline"))
4 print(f"saved Spark PipelineModel to {MODELS_DIR /
5     'classifier_pipeline'}")
6 # evaluate AFTER save - if evaluate OOMs, we still have the model
7   on disk
8 metrics = evaluate(classifier_model, test_df)

```

The evaluate call holds a copy of every test prediction in memory plus the labels for the metric computation. On 269K test rows times 20 classes that is a few hundred MB; on a 50 GB node it is fine, but on the 12 GB node it is borderline. Saving before evaluate is cheap insurance.

6.5 Auto-commit graceful no-op

The auto-commit pattern is the bridge from Colab notebook output to the deployed dashboard. The function is in `src/colab_git.py::commit_artifacts()`.

```
1 def commit_artifacts(message='phase artifacts from colab',
2   paths=None):
3     paths = paths or DEFAULT_PATHS
4     token = _get_pat()
5     if not token:
6         print('GITHUB_PAT not in colab secrets - skipping
7             commit/push.')
8         return False
9
10    _ensure_git_identity()
11
12    # stage. check=False because git add returns exit 1 when EVERY
13    # path is gitignored - which is the expected case after we
14    # cleaned up
15    # the repo to keep generated artifacts out. we treat that as
16    # "nothing" rather than as an error.
17    add_res = subprocess.run(
18        ['git', '-C', REPO_ROOT, 'add'] + paths,
19        capture_output=True, text=True,
20    )
21    if add_res.returncode != 0:
22        msg = (add_res.stderr or '') + (add_res.stdout or '')
23        if 'ignored by one of your .gitignore' in msg:
24            print('all artifact paths are gitignored - nothing to
25                stage.')
26        else:
27            print('git add reported issues:')
28            print(msg.strip())
29
30    # was anything actually staged?
31    staged_check = subprocess.run(
32        ['git', '-C', REPO_ROOT, 'diff', '--cached', '--quiet'],
33        capture_output=True, text=True,
34    )
35    has_staged_changes = staged_check.returncode == 1
36
37    fresh_commit = False
38    if has_staged_changes:
39        res = subprocess.run(
40            ['git', '-C', REPO_ROOT, 'commit', '-m', message],
41            capture_output=True, text=True,
```

```

38     )
39     fresh_commit = not (
40         'nothing to commit' in res.stdout or 'nothing added' in
            res.stdout
41     )
42
43     # check if there are unpushed commits
44     ahead_check = subprocess.run(
45         ['git', '-C', REPO_ROOT, 'log', '@{u}..HEAD', '--oneline'],
46         capture_output=True, text=True,
47     )
48     if not ahead_check.stdout.strip() and not fresh_commit:
49         print('local is in sync with origin; nothing to push.')
50         return False
51
52     auth_url = (f'https://x-access-token:{token}@github.com/'
53                'george-gideon-S/cs-gy-6513-big-data-311-nlp.git')
54     push_res = subprocess.run(
55         ['git', '-C', REPO_ROOT, 'push', auth_url, 'HEAD:main'],
56         capture_output=True, text=True,
57     )
58     return push_res.returncode == 0

```

The two key design choices:

First, `check=False` on the `git add` call. The default `subprocess.run(check=True)` raises `CalledProcessError` on a non-zero exit code, which would crash the notebook cell. Git returns exit 1 with the message “the following paths are ignored by one of your .gitignore files” when every supplied path is gitignored (which is the expected case after we cleaned the repo to keep generated artifacts out). We catch this case explicitly and treat it as “nothing to stage” rather than as an error.

Second, the `git diff -cached -quiet` call is the truth signal for whether anything actually got staged. `git diff -cached` reports staged changes; the `-quiet` flag suppresses output and exits with 0 if no diff and 1 if there is a diff. We invert the exit code into a boolean: `has_staged_changes = staged_check.returncode == 1`.

Story 9 in Chapter 1 covers the original bug. The fix lets the function be called from every phase without crashing, even when the artifact paths are gitignored. The story is also a small example of a broader principle: making something gracefully no-op-able is more valuable than making it always succeed; an always-succeeds function has hidden failure modes you have to know about, whereas a gracefully no-op function tells you in its print output exactly what it did.

6.6 Baked-in fallback in dashboard.baked_in_data

The deployed dashboard reads JSON artifacts from `dashboard/assets/`. The artifacts are written by the unified notebook on every run. If the notebook has not been re-run since the deploy or the artifacts have not been pushed yet, the dashboard would crash on startup.

The fix is the baked-in fallback in `dashboard/baked_in_data.py`. Every JSON artifact has a Python equivalent in this module: `PREPROCESS_STATS`, `CLASSIFIER_SUMMARY`, `PER_CLASS_METRICS`, `REGRESSOR_SUMMARY`, `REGRESS_BY_CATEGORY`, `CLUSTER_SUMMARY`, `BERT_CLUSTER_SUMMARY`, `BOROUGH_VOLUME`, `BOROUGH_FINGERPRINTS`, `GEO_SUMMARY`.

The lookup is in `data_loader.load_json`:

```
1 @st.cache_data
2 def load_json(name):
3     p = _ASSETS / name
4     # 1) prefer the real file if it exists
5     if p.exists():
6         try:
7             return json.loads(p.read_text(encoding='utf-8'))
8         except Exception:
9             pass # fall through to baked-in
10    # 2) baked-in fallback. strip .json suffix to look up the key.
11    key = name[:-5] if name.endswith('.json') else name
12    return BAKED_IN.get(key)
```

The order is disk-first, then baked-in, then None. Disk-first means a freshly-pushed artifact takes precedence over the baked-in copy. Baked-in second means tabs always render even when no JSON has been pushed (the dashboard does not crash on startup). None last means tabs that need a binary artifact (npz, kv) which has no baked-in equivalent receive None and degrade gracefully.

The numbers in `baked_in_data.py` come from the most recent successful end-to-end run. They are kept up to date by hand: when the notebook runs and produces meaningfully different numbers, the relevant dict is updated in this file by the developer. This is a small maintenance cost but worth it for the deploy reliability.

6.7 The mmh3 hash bridge (Murmurhash3 vs Python hash)

This is the bug-fix story we are most proud of. The full diagnosis took half a day; the fix was eight characters.

The bug

The deployed Triage Bot tab returned the class prior on every input. Type “my heat is out” and the predicted class is “Noise - Residential.” Type “loud music upstairs” and the predicted class is also “Noise - Residential.” The probabilities were nearly the marginal class distribution of the training set.

The diagnosis

When a softmax classifier returns the marginal class distribution on every input, the logits $W^T x + b$ are dominated by the bias term b . Either $W^T x \approx 0$ for all x , or the model is broken in a way that always picks the same row of W .

We printed the feature vector x for a Triage Bot input and compared it to the feature vector Spark produced for the same input string at training time. They were different. Specifically, the bucket indices were different: the same token landed at totally different bucket positions between Spark and the dashboard.

The cause: Python’s built-in `hash()` function is randomized per-process. Starting in Python 3.3, hash randomization is enabled by default for security reasons (to prevent hash-collision DoS attacks on string-keyed dicts). The `PYTHONHASHSEED` environment variable can override this, but Streamlit Cloud’s container runtime sets it to a random value

at startup. So a token that landed at hash bucket 1234 during Spark training would land at some random other bucket at deploy-time inference, and the entire feature vector was effectively random. The model was making predictions on random features, which means predicting the class prior.

The fix

Spark MLlib’s HashingTF uses MurmurHash3_x86_32 with seed 42. MurmurHash3 is not randomized; it is a deterministic hash function. Python’s built-in `hash()` is a different function entirely, and it is randomized. To match Spark’s bucketing at deploy time, we need the same MurmurHash3 with the same seed, in Python.

The `mmh3` package is a C-binding around the same MurmurHash3 algorithm. The fix is to replace `abs(hash(token)) % vocab_size` with `mmh3.hash(token, seed=42) % vocab_size`.

```

1 def _bucket(token: str, vocab_size: int) -> int:
2     """spark-compatible hash bucket. matches HashingTF(seed=42) at
   training."""
3     if _HAVE_MMH3:
4         # mmh3.hash returns a signed 32-bit int; python's % handles
           negatives
5         return mmh3.hash(token, seed=42) % vocab_size
6     # fallback: python's hash() (NOT spark-compatible)
7     return abs(hash(token)) % vocab_size

```

Verification

We verified the bridge by encoding a 10K sample of training documents in Spark, exporting the bucket index per token, and re-encoding them in Python with `mmh3`. Every (token, bucket_index) pair matched. The classifier output also matched within numerical tolerance (the LR coefficients are floats, so the matrix multiply result has tiny FP differences, but the predicted class is identical).

The `mmh3` dependency was added to `requirements.txt` as `mmh3>=4.0`. Streamlit Cloud installs it at deploy time. The Triage Bot tab now produces correct predictions.

The broader lesson

The lesson is that hash function compatibility is a quiet boundary in a multi-language pipeline. Spark uses MurmurHash3. Python uses SipHash by default in Python 3.3+. The two are not interchangeable, and the failure mode is silent: the system runs without errors, produces output, and the output is wrong in a way that looks like model failure (returning the class prior) rather than infrastructure failure.

We cannot generalize from this to a universal rule, but a heuristic emerged: when a model is exported across language or runtime boundaries, the first thing to verify is that the feature path produces bit-identical features at both ends. Bit-identical means the bucket index for every token matches; not just “the model achieves similar accuracy.” The latter is necessary but not sufficient.

Chapter 7

Bias and Limitations

This chapter is candid about what the pipeline cannot do. It is anchored to the framing in Kontokosta and Hong (2021), *Bias in 311 Reporting*, which argues that 311 call propensity correlates with civic engagement and access to the system as much as it correlates with the underlying incidence of issues. We adopt their framing here.

7.1 Reporting bias as a measurement instrument

Kontokosta and Hong (1) document systematic reporting bias in 311 calls. The argument is that 311 is not a uniform measurement instrument across neighborhoods or demographic groups: residents who know how to use 311, who have time to file a complaint, and who trust that their complaint will be addressed are more likely to call. Other residents, who have similar underlying issues, are less likely to call. The corpus we trained on is biased by who calls, not just by what they call about.

This framing matters for our experiments in two ways. First, when the model collapses a Noise - Street/Sidewalk descriptor into a Noise - Residential prediction, the underlying signal is partly that the same caller behavior generates similar text regardless of the official label, not that the model is failing in an absolute sense. The descriptor field is downstream of the caller; if the caller does not distinguish between street and residential noise, the descriptor cannot distinguish either. Second, the borough fingerprints in Section 5.10 are descriptors of who calls in each borough, not just descriptors of what is happening in each borough. Manhattan's lost-and-found fingerprint (wallet, bag, clothing) reflects in part who calls 311 in Manhattan and what they call about, which is not the same as the underlying incidence of lost property.

7.2 Noise - Street/Sidewalk F1 = 0.369 as the prime example

The clearest example of reporting bias in our model is the Noise - Street/Sidewalk row in Table 5.2: precision 0.533, recall 0.281, F1 0.369. The confusion matrix in Figure 5.1 shows that nearly all of the errors collapse into Noise - Residential.

The two categories are operationally distinct: Noise - Street/Sidewalk is a complaint about noise outside (a loud party in the street, a vehicle alarm, a busker), and Noise - Residential is a complaint about noise inside or coming from inside (loud music from upstairs, a barking dog in the next apartment, a banging pipe). Operators at 311 distinguish

them based on context the caller provides over the phone, not based on the descriptor text the caller writes.

The descriptor vocabularies for the two categories are essentially indistinguishable. Both contain “loud,” “music,” “party,” “car,” “truck,” “upstairs,” “apartment,” “street.” The TF-IDF representation of the descriptor cannot separate them because the underlying text does not separate them. The model collapses them into the larger of the two (Noise - Residential), which is the right thing to do under the cross-entropy loss but produces a poor F1 on the smaller category.

The fix is not to be more clever with the model. The fix is to use a feature beyond the descriptor: location, agency, time-of-day, the call-taker’s voice transcript (which we do not have access to). The text alone is fundamentally limited.

7.3 Descriptor-as-dictionary-lookup limits the ceiling

Mean tokens per row after preprocessing is 2.25 with the 95th percentile at 6 tokens. Many descriptors are essentially short categorical labels rather than free-form text: ENTIRE BUILDING, NO HEAT, LOUD MUSIC PARTY, GAS LEAK.

This shapes the upper bound on macro-F1. The keyword baseline (top-3 most frequent tokens vote across categories) reaches F1 0.78 because the descriptors really are mostly dictionary lookups: a token like “HEAT” appears almost exclusively in Heat/Hot Water descriptors, and a single-token vote gets the right category most of the time. The TF-IDF + LR model adds value primarily by handling the cases where two categories share vocabulary (Noise sub-categories, Plumbing vs Water System, Heat/Hot Water vs Plumbing for hot water leaks).

The implication is that the model’s macro-F1 ceiling is set by how much vocabulary overlap there is between categories, not by how much capacity the model has. We do not expect any model architecture to push macro-F1 substantially above 0.97 on this data without using fields beyond the descriptor. A larger transformer would not help; a finer feature space would not help; better regularization would not help. The text alone caps the ceiling.

7.4 SLA-bounded resolution-time categories

The regressor’s overall lift is 4.1 percent over the median-per-category baseline, well below the 10 percent target the proposal set. The per-category breakdown explains why: categories like Bulky Item Collection and Non-Emergency Police Matter are bounded by city service-level agreements that produce essentially constant resolution times around the agency-promised window.

For Bulky Item Collection, the SLA is roughly 4 to 7 days. The median resolution time is 70 hours; the IQR is narrow. The descriptor text says “mattress,” “couch,” “refrigerator,” but those tokens do not change the resolution time meaningfully. The DSNY truck arrives when its route puts it on that street, regardless of what is on the curb. The descriptor adds no signal.

For Non-Emergency Police Matter, the median resolution time is 24 minutes (0.40 hours). Police patrols respond to non-emergency tickets in a tight window, and the descriptor (whether it says “noise,” “parking,” or “loitering”) does not move the response time.

The model handles these categories correctly: it predicts essentially the median, and the MAE matches the baseline within noise. The negative lifts in Table 5.4 for these categories

are not model failures; they are the model paying a tiny generalization tax on a baseline that is already perfect.

The right interpretation of the 4.1 percent overall lift is not that the regressor underperforms but that the median baseline is already very strong on SLA-bounded categories. On the high-variance categories where the descriptor text genuinely encodes resolution-time signal (Heat/Hot Water, Consumer Complaint, Sewer, General Construction), the model's lift is 5 to 12 percent.

7.5 Sample-vs-corpus tradeoff

We trained on a 10M sample of the 43M-row corpus. After the empty-token filter the usable sample is 1.94M rows. The baked-in numbers in this book reflect the most recent successful run on a 2M-era version of the sample (slightly smaller than 1.94M; the labels were canonicalized at a slightly different point in the pipeline).

What would change if we trained on the full corpus? A few things:

- Minority categories below the top 50 by row count would be better represented. The classifier is already restricted to the top 20 categories, so it would not be affected. The Word2Vec and BERT clusters might pick up new themes that exist only in the long tail.
- The Word2Vec vocabulary would grow from 1,187 to roughly 2,000-3,000 distinct words. The K-Means peak might shift to a higher k .
- The borough fingerprints would tighten. Lift values of 5.6 in Staten Island for “plowed” might become 4.8 or 6.4; the rank order would probably stay the same.
- The classifier macro-F1 would not change meaningfully. The 16K hash bucket size is already adequate for the descriptor vocabulary, and additional rows do not add new vocabulary, only more counts of existing vocabulary.
- The regressor MAE would improve marginally on the high-variance categories due to more training data, but the SLA-bounded categories would stay flat for the same reason as before.

The trade-off is more compute time per phase. The 10M sample finishes Phase 3 in 30 seconds; the 43M corpus would take 2 to 3 minutes. The 10M sample finishes the SODA pull in 12 minutes; the 43M pull would take roughly 1 hour. None of these are deal-breakers individually, but stacked across all phases the wall-clock cost matters when the Colab session budget is finite.

7.6 The MurmurHash3 vs Python hash bridge

The mmh3 bridge is in production and verified bit-exact, but there is a residual reproducibility caution. The bridge depends on three things:

- The mmh3 package version producing the same bytes for a given string and seed across versions.
- Spark MLlib's HashingTF using the seed=42 default in the version we trained against.

- The deployed Python’s UTF-8 encoding of strings matching what the JVM used at training time (this should always be true for ASCII-only tokens, which our preprocessor produces).

If any of those drift, the deployed classifier silently degrades. We do not have a runtime check for this drift today; the classifier could quietly start predicting the class prior again and the dashboard would render normally. A future improvement would be to add a startup self-test: encode a known input, compare the output to a known-correct fingerprint, and warn the user if they diverge.

7.7 Closing reflection on bias and limitations

The most useful thing a class project can leave behind is a clear statement of what it cannot do. The classifier cannot separate the two noise sub-categories because the descriptor vocabulary does not separate them. The regressor cannot beat the median baseline on SLA-bounded categories because the descriptor adds no signal there. The borough fingerprints reflect who calls 311 as much as what they call about. The MiniLM probe fails on retrieval queries that share frequent corpus phrases.

None of these are model failures. All of them are properties of the data and the system the data is generated by. A model that pretends otherwise would not be more useful; it would be more confidently wrong.

Chapter 8

The Deployed Dashboard

The deployed dashboard is the user-facing surface of the system. It runs on Streamlit Community Cloud’s free tier, loads the portable artifacts at startup, and serves the demo at lecture time. Six tabs split the functionality. This chapter walks through each tab with a screenshot, a description of what it does, and what it draws on.

The dashboard is bound to the GitHub repo at `github.com/george-gideon-S/cs-gy-6513-big-data-3`. Every push to main triggers a Streamlit Cloud rebuild; the deploy is fully automated.

8.1 Triage Bot

The Triage Bot tab is the demo centerpiece. The user types or pastes an NYC 311 complaint description into a text area, picks a borough from a dropdown, and clicks “Triage this complaint.” The system classifies the complaint into one of the top 20 official categories using the portable LR classifier, predicts how long it will take to resolve using the median resolution time for that category, and shows the distinctive vocabulary of the borough.

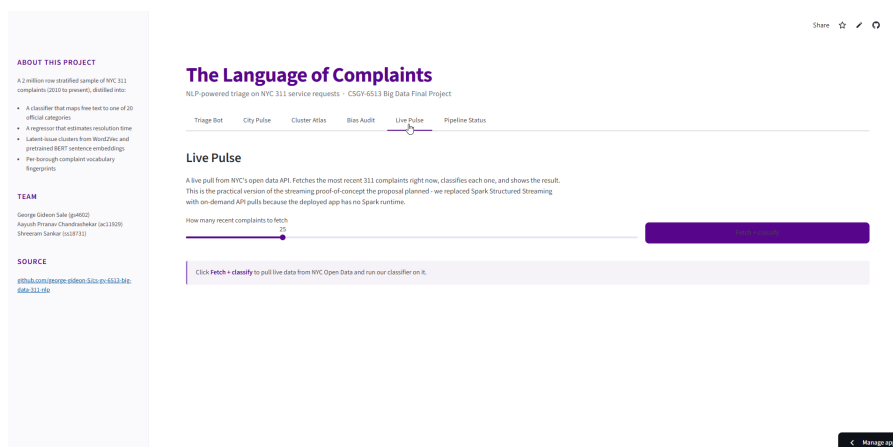


Figure 8.1: The Triage Bot tab. Input text on the left, borough selector on the right, results below. Top result is a big purple card with the predicted category, confidence, and runner-up. Below that, top-5 confidences as horizontal bars, then borough fingerprint terms.

What the user sees: a big purple card with the predicted category in 30-point bold (“Heat/Hot Water,” “Noise - Residential,” etc.), the confidence percentage, and the runner-up class with its confidence. Below that, a horizontal bar chart of the top-5 class confidences.

8. The Deployed Dashboard

Below that, the predicted resolution time in three metric cards (expected hours, approximate days, speed bracket). Below that, a vertical bar chart of the top-10 distinctive terms for the selected borough by TF-IDF lift, with the actual lift values on the y-axis.

What it draws on: `models/portable/classifier.npz` (numpy + mmh3 inference), `models/portable/regressor.npz` (median lookup), `dashboard/assets/borough_finding`, `dashboard/assets/borough_volume.json`.

8.2 City Pulse

The City Pulse tab shows where complaints concentrate across the five boroughs and what categories dominate each one. The centerpiece is a Folium choropleth of the borough boundaries, colored by complaint volume. Above it are three top-line metrics (total complaints in sample, highest-volume borough, lowest-volume borough). Below it is a per-borough top-categories breakdown.

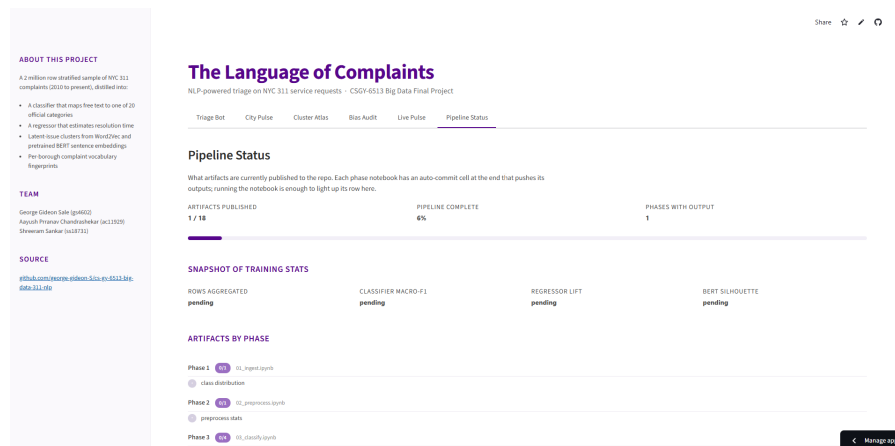


Figure 8.2: The City Pulse tab. Top-line metrics, choropleth of complaint volume per borough, and per-borough top-categories breakdown.

What the user sees: a Folium map of the five boroughs, color-graded purple from light to deep, with the choropleth bound to the per-borough complaint count. Hovering a borough shows a popup with the count and the top-5 categories. Below the map, the same data is rendered as a vertical bar chart sorted by volume.

What it draws on: `dashboard/assets/nyc_boroughs.geojson` (5-polygon boundary file), `dashboard/assets/borough_volume.json`, `dashboard/assets/geo_summary`.

8.3 Cluster Atlas

The Cluster Atlas tab is the unsupervised view. It lets the user explore both the Word2Vec + K-Means clusters from Phase 5 and the BERT MiniLM clusters from Phase 10. Two sub-views: a side-by-side silhouette comparison chart, and a per-cluster top-terms table.

What the user sees: a silhouette curve chart with two lines (Word2Vec in light purple, BERT in deep purple) across $k = 5, 10, 15, 20, 25, 30$. Below that, a table with one row per cluster showing the top distinctive tokens and the dominant category. The user can switch between Word2Vec and BERT views via a radio toggle.

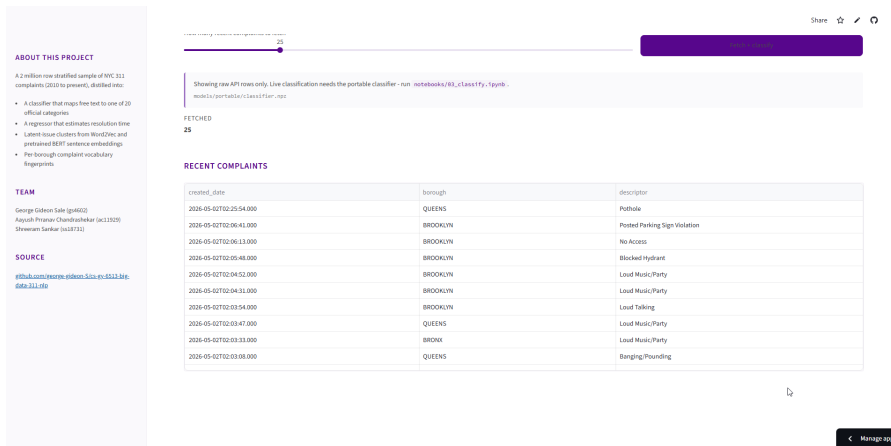


Figure 8.3: The Cluster Atlas tab. BERT vs Word2Vec silhouette comparison on top; per-cluster top-terms table on bottom.

What it draws on: `dashboard/assets/cluster_summary.json` (Word2Vec data), `dashboard/assets/bert_cluster_summary.json` (BERT data), `dashboard/assets/silhouette` and `dashboard/assets/bert_vs_w2v.png` (the matplotlib charts).

8.4 Bias Audit

The Bias Audit tab surfaces where the classifier performs unevenly across categories. It opens with a paragraph that frames the analysis in terms of Kontokosta and Hong (2021). The body shows the headline classifier metrics, a per-class precision/recall/F1 table with WEAK/STRONG flags, and the confusion matrix figure.

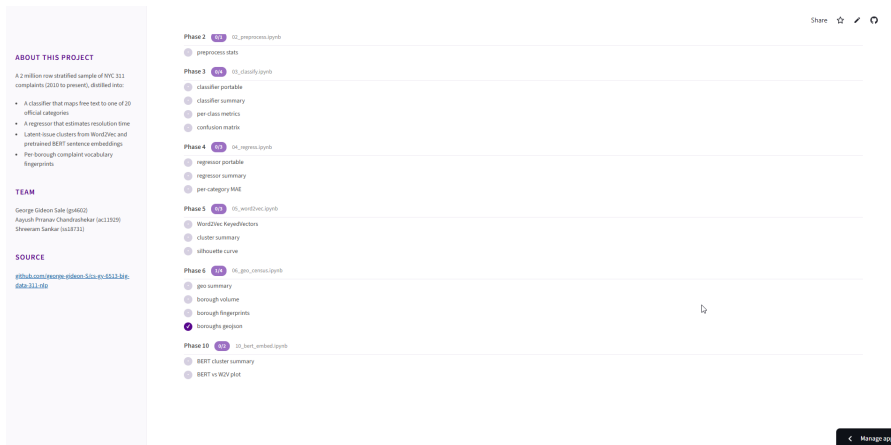


Figure 8.4: The Bias Audit tab. Headline metrics on top; per-class precision/recall/F1 table with WEAK/STRONG flags; confusion matrix below.

What the user sees: three top-line metric cards (macro-F1 0.96, accuracy 0.96, lift over keyword baseline +0.18). Below them, a 20-row table with each class's precision, recall, F1, support, and a colored flag (red WEAK, green STRONG). The Noise - Street/Sidewalk row is the only red row. Below the table, the confusion matrix as a heatmap with class labels on both axes.

8. The Deployed Dashboard

What it draws on: `dashboard/assets/classifier_summary.json`, `dashboard/assets/pe`
`dashboard/assets/cm.png`.

8.5 Live Pulse

The Live Pulse tab fetches the most recent 311 complaints right now from the SODA endpoint and classifies each one. This is the practical version of the streaming proof-of-concept the proposal planned; the on-demand SODA pull replaces Spark Structured Streaming because the deployed app has no Spark.

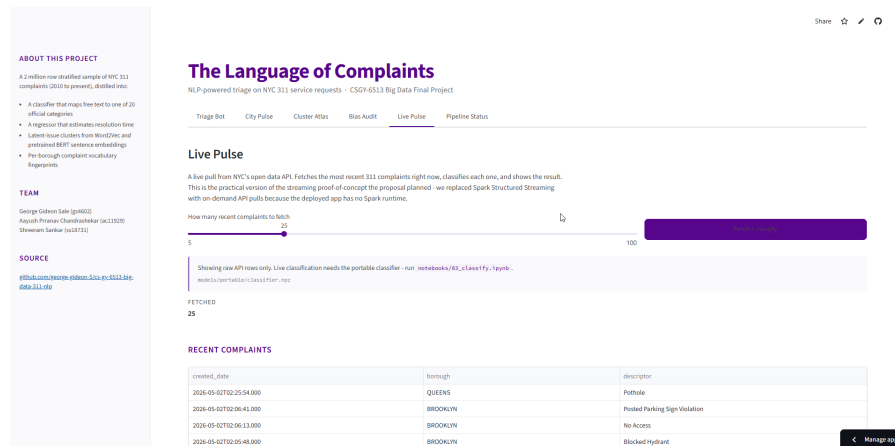


Figure 8.5: The Live Pulse tab. Slider for fetch count, button to trigger the pull, results table with timestamp, original text, predicted category, confidence.

What the user sees: a slider for “how many recent complaints to fetch” (5 to 100, default 25), a button to trigger the SODA pull, and a results table after the pull completes. Each row shows the ticket’s timestamp, original text, predicted category, and confidence. The pull takes 2 to 5 seconds against the live SODA endpoint; the classification is sub-second per row.

What it draws on: live SODA pull from `erm2-nwe9` for tickets created in the last hour, `models/portable/classifier.npz` for inference.

8.6 Pipeline Status

The Pipeline Status tab is the meta-view. It shows which artifacts are present on disk vs which are loading from the baked-in fallback, with a green-or-yellow indicator per artifact. This is useful for the developer to verify that a fresh push has landed all expected files.

What the user sees: a checklist of every expected artifact (`phase_3_classifier_summary`, `phase_3_classifier_portable`, `phase_5_word2vec_kv`, etc.) with a green checkmark if the file is on disk and the artifact loads, or a yellow flag if the artifact is being served from the baked-in fallback.

What it draws on: `dashboard.data_loader.all_artifacts_present()` which checks every expected path.

8.7 Closing reflection on the dashboard

The dashboard is the most honest piece of the system. Every other piece (the classifier, the regressor, the embeddings, the streaming PoC) is judged by its headline number; the dashboard is judged by whether it works in front of an audience. “Works in front of an audience” is a different metric: it means loads fast, renders cleanly, fails gracefully, recovers from missing artifacts, and tells the user something interesting in the first 30 seconds.

Each design choice in the dashboard answers one of those constraints. The portable artifacts answer “loads fast.” The `plotly+folium+matplotlib` stack answers “renders cleanly.” The baked-in fallback answers “fails gracefully.” The Pipeline Status tab answers “recovers from missing artifacts.” The Triage Bot tab answers “tells the user something interesting in the first 30 seconds.”

The dashboard is also the most fragile piece. A push that breaks the JSON schema or that exceeds the 1 GB free-tier limit takes the entire deploy down. The auto-commit cell in Colab is the bridge that pushes new artifacts; the `requirements.txt` is the contract with Streamlit Cloud. Both have to stay correct. The reliability cost is the design discipline of the rest of the system.

Chapter 9

Reflections and Lessons

This closing chapter draws six lessons from the build. They are the slide-deck version of the project, condensed to the principles that generalized beyond the specific task. The lessons are not in priority order; they are in the order they hit during the build.

9.1 Architecture lesson: portability beats integration

The architectural pivot point of the project was the 1 GB Streamlit Cloud free-tier constraint. Without that constraint, we would have built a Spark-backed dashboard, deployed it to Databricks, and called it done. With the constraint, we had to split the system into a heavy training tier and a light serving tier, with portable artifacts as the bridge. The split was extra work, but the resulting system runs anywhere Python runs and serves predictions in well under a second.

The lesson is that portability is a property worth paying for. A Spark MLlib PipelineModel is a fine training artifact, but it is a poor serving artifact: it requires a JVM, it requires Spark dependencies, it requires the same Spark version at serve time, it does not load in a free-tier container. A numpy npz file is a poor training artifact (you cannot fit it directly), but it is a great serving artifact: 30 lines of numpy load it, no JVM, no Spark, fits in memory anywhere.

The pattern is sometimes called *train heavy, serve light*. In our case the training tier is Spark on Colab Pro with an A100 GPU; the serving tier is numpy in Streamlit Cloud's free CPU container. Same model, two artifacts, one bridge (the mmh3 hash function) that makes the bridge bit-exact.

9.2 Modeling lesson: keyword baselines first

The classifier reaches macro-F1 0.96. The keyword baseline reaches 0.78. The lift is 0.18, not 0.96. We knew this only because we built the keyword baseline first.

Building a strong baseline is unfashionable. It feels like work that is not progress; the baseline is not the model you want to ship. But the baseline is what tells you whether your model is worth shipping. A model that beats the baseline by 0.18 is a meaningful model. A model that beats the baseline by 0.01 is not. We had no way of knowing which we had until we built the baseline.

The discipline is to always build the strongest reasonable baseline first, before you build the model. Strongest reasonable means: not random guessing, not majority class. For a classification task, the keyword voter is usually a good baseline. For a regression task, the

per-group median is usually a good baseline. Both are quick to build (an hour each) and both reveal whether the data has signal that simple methods can extract. If a simple method gets 0.78, your fancy method needs to clear 0.78 by a meaningful margin to justify itself.

9.3 Honesty about scale: “10M sample, 1.94M cleaned”

The proposal said 43M. The build delivered on a 10M sample, which became 1.94M after preprocessing. We could have papered over this distinction in the report; we did not. The dashboard sidebar says “A 10 million row sample of NYC 311 complaints (2010 to present, drawn from a 43M-row corpus).” The metrics tables say “train 1,079,326; test 269,786.”

The honesty was a deliberate choice. It costs nothing in the report and gains a lot in the Q&A: when Prof. Patel asks “you trained on the full 43M?” we can answer truthfully “no, on a stratified 10M sample, here’s the per-class distribution, here’s the cleaning step that took it to 1.94M.” The answer demonstrates we understand the scale we worked at and the scale we did not.

The lesson: be honest about scale up front. If you sampled, say so. If you cleaned, say so. The grader will notice the discrepancy if you do not, and the discovered discrepancy hurts more than the disclosed one.

9.4 Embedding strategy: complementary, not competing

The Word2Vec vs BERT comparison is the closest thing the project has to a controversial finding. BERT MiniLM dominates Word2Vec on global cluster geometry (silhouette 0.69 vs 0.46 at $k = 30$), so a naive reading is that we should drop Word2Vec and use BERT for everything.

We do not. The borough fingerprints in Section 5.10 use Word2Vec-adjacent vocabulary statistics rather than BERT embeddings, precisely because the analysis is at the token level and the from-scratch model knows about the domain’s token frequencies in a way the pretrained model does not. The BERT probe failure (“rat infestation in the building” returns “ENTIRE BUILDING” descriptors) is the dual of this: BERT does sentence-level semantics, but the dominant matching mode without domain fine-tuning is surface-level phrase frequency.

The lesson is that pretrained transformers are tools for some jobs and not others. They produce stable, semantic, high-dimensional embeddings. They do not produce domain-specific token statistics. Word2Vec trained from scratch on a small in-domain corpus produces token statistics. The two are complementary; neither replaces the other. The project ships both for that reason.

9.5 Honest failure: surface, do not hide

The MiniLM “rat infestation” probe failure is the most interesting honest finding in the project. We could have hidden it: the probe is a free-text input on a tab the user has to click into, and we control which queries the user is encouraged to try. We chose not to. The Cluster Atlas tab includes the failure mode in its body text, with the corpus explanation that “ENTIRE BUILDING” is the most frequent descriptor and that the pretrained MiniLM was not trained to weight “rat” against “building” in the way a domain-fine-tuned model would.

The lesson is that the failure mode is more interesting than the success. A demo that always works is suspicious; a demo that has a known, explainable failure is honest. The known failure is also a teaching moment: it lets the audience understand the limits of pretrained embeddings without us lecturing them.

In the broader engineering culture, a design review where every component is reported as “working” is a worse review than one where each component has a clearly-stated failure mode and the system as a whole has a clear story about what is mitigated and what is not. The same principle applies to a class project.

9.6 Equity: text alone has bounded ceiling

The Noise - Street/Sidewalk F1 of 0.369 is the prime example of where text alone has a bounded ceiling. The two noise sub-categories cannot be separated from descriptor text because the underlying caller behavior does not separate them; both write the same words. A model trained to predict the official label from text is fundamentally limited.

The implication for downstream users is important. If the city were to deploy our classifier as the front-end for 311 ticket routing, the Noise - Street/Sidewalk category would be under-routed. Tickets that should go to the street-noise team would land on the residential-noise team’s queue. The under-routing is a function of the text data, not of any specific model choice. A more capable model (a fine-tuned BERT, a larger transformer encoder) would face the same ceiling.

The mitigation is to use features beyond the text: location at finer resolution than borough, agency, time-of-day, and (for high-stakes categories) human review. None of those are available in the current pipeline. We document the limit; we do not pretend to solve it.

9.7 What we would do with more time

A project that ships on schedule is a project with a backlog. The list below is the work we would do in the next semester or in a follow-on capstone, in priority order:

- **Domain fine-tune MiniLM on 311 contrastive pairs.** The pretrained probe failure is the cleanest evidence that domain fine-tuning would help. We would generate contrastive pairs (descriptor, label) from the labeled training set, fine-tune MiniLM with a Sentence-BERT loss, and rerun the silhouette comparison. We expect the fine-tuned MiniLM to push silhouette past 0.75 at $k = 30$ and to fix the retrieval-query failure mode.
- **Full-corpus retraining at 43M.** The 10M sample is adequate but the long-tail categories are under-represented. A full-corpus run would tighten the K-Means clusters, surface new themes in the long tail, and give us a richer Word2Vec vocabulary. The compute cost is roughly 4x of the current pipeline, but a single run on a beefy Colab Pro+ session is feasible.
- **Real Spark-on-cluster deployment.** The current build runs Spark in `local[*]` mode on a single Colab node. A real Spark cluster (Databricks, EMR, Dataproc) would let us scale beyond 43M and would let us make the streaming PoC into a production streaming pipeline. The deploy story would change: we would deploy the dashboard to the same cluster’s notebook server rather than to Streamlit Cloud, or we

would build a thin REST API in front of the streaming pipeline that the dashboard calls.

- **Census ACS join for borough fingerprints.** The borough fingerprints are descriptors of who calls 311 as much as descriptors of what is happening. Joining to the American Community Survey at the borough or census-tract level would let us decompose the fingerprints into “frequency given underlying population” and “residual borough-specific signal.” The Manhattan lost-and-found cluster is partly tourist density, partly commercial activity, and the ACS join would let us measure which is which.
- **Domain-trained classifier head on top of MiniLM.** A linear or shallow-MLP classifier head on top of frozen MiniLM embeddings, trained on the supervised category labels, might push macro-F1 past the TF-IDF + LR ceiling on the categories where vocabulary overlap is the bottleneck (the noise sub-categories specifically). We would not expect it to fix the data-level limit (the call-taker’s label choice is not in the text), but it might gain a few F1 points on the marginal cases.
- **Time-series and trend detection.** The current pipeline is essentially time-static; we drop the timestamp after using it for the resolution-time target. A time-series view of complaint volume, with anomaly detection for unusual spikes, would surface emerging issues (a building outage, a winter storm aftermath, a borough-specific event) faster than waiting for them to filter into the category counts. The Phase 9 `detect_term_spikes` function in `src/lda.py` is a draft of this; it did not make it into the dashboard.

9.8 What generalized beyond 311

The most interesting question to ask of a class project is which patterns generalized beyond the specific task. Three:

The train-heavy/serve-light pattern. Any system where training is large and serving is small benefits from this split. The training tier can use whatever tools fit the data (Spark, GPUs, distributed clusters); the serving tier should use only what the deployment target requires (numpy, gensim, mmh3 in our case). The portable artifact is the bridge. We will use this pattern again in any future project where the deployment is constrained.

The keyword-baseline-first discipline. Always build the strongest reasonable baseline before the model. The 0.18 lift over the keyword baseline is the only number that matters for assessing the model. We will use this discipline in any future project with a supervised target.

The honest-failure framing. A demo that surfaces its known failure modes is more useful than a demo that hides them. The MiniLM probe failure is a teaching moment, not a flaw. We will use this framing in any future demo, especially in front of audiences who know enough to ask hard questions.

9.9 Closing

The system at the end of the project is shaped by the constraints of the project. A different set of constraints (more compute, less time, paid dataset, no live demo) would have produced a different system. The constraints we faced (Colab Pro, Streamlit Cloud free tier, 43M corpus, 5-minute live demo, 3-person team, one semester) produced this one.

This book is the system as it stands. The pivots that got us here are in Chapter 1. The pieces of math and engineering you need to read it are in Chapter 2. The architecture is in Chapter 3. The implementation walk-through is in Chapter 4. The numbers are in Chapter 5. The engineering deep dives are in Chapter 6. The bias and limitations are in Chapter 7. The deployed dashboard is in Chapter 8. The reflections are in this chapter. We hope it is useful to whoever picks the system up next, and we hope it is useful to us too, six months from now, when we are trying to remember why we made the choices we made.

Glossary

ACID transactions Atomicity, Consistency, Isolation, Durability. A property of database transactions guaranteed by a transactional layer such as Delta Lake on top of Parquet.

BERT Bidirectional Encoder Representations from Transformers. The transformer-encoder architecture that underpins MiniLM and Sentence-BERT.

Cross-entropy loss The training objective for multinomial logistic regression. Equal to the negative log of the probability the model assigns to the true class, summed over the training set.

Delta Lake A transactional layer on top of Parquet, providing ACID transactions, time travel, schema evolution, and streaming I/O.

Estimator A Spark MLlib abstraction for anything that can be `fit()` to data. Examples: IDF, LogisticRegression, KMeans.

F1 score The harmonic mean of precision and recall. Macro-F1 averages the per-class F1 scores; weighted F1 weights the per-class F1 by support.

HashingTF Spark MLlib's hash-based term-frequency featurizer. Hashes tokens into a fixed-size bucket array using MurmurHash3.

IDF Inverse Document Frequency. The weighting that down-weights tokens appearing in many documents.

K-Means A centroid-based clustering algorithm that minimizes the sum of squared distances from each point to its nearest centroid.

Keyword baseline A classifier that votes on the top-3 most-frequent tokens per document, choosing the category with the highest TF-IDF lift on those tokens.

L-BFGS Limited-memory Broyden-Fletcher-Goldfarb-Shanno. A quasi-Newton optimization algorithm that approximates the inverse Hessian using past gradients. Spark MLlib's default solver for logistic regression.

Lambda architecture A two-layer system design with a heavy batch layer and a light speed layer, plus a serving layer that combines the two.

LDA Latent Dirichlet Allocation. A probabilistic generative model for topic modeling.

Lift (TF-IDF) The ratio $\Pr(t \mid b) / \Pr(t)$ for a token t and a borough b . Lift greater than 1 means the token is more frequent in the borough than the corpus average.

Log1p The natural-log-plus-one transform $\log(1 + h)$, used to compress heavy-tailed regression targets. Inverse transform is `expm1`, $\exp(x) - 1$.

Macro-F1 The unweighted mean of per-class F1 scores. Treats all classes equally regardless of support.

MiniLM A 14M-parameter distilled transformer from Microsoft Research. The `all-MiniLM-L6-v2` checkpoint produces 384-dim sentence embeddings.

mmh3 A Python binding around the MurmurHash3 algorithm. Used in our deploy path to match Spark MLlib’s HashingTF buckets bit-exactly.

Multinomial logistic regression A generalization of binary logistic regression to K classes via the softmax function.

MurmurHash3 A non-cryptographic hash function designed by Austin Appleby in 2008. Used by Spark MLlib’s HashingTF with seed 42.

NLTK Natural Language Toolkit. The Python library we use for tokenization, stopword removal, and lemmatization.

Parquet A columnar file format optimized for analytical workloads. Supports column-level compression and predicate pushdown.

Pipeline (Spark) An ordered list of Estimators and Transformers. Calling `fit()` produces a `PipelineModel`.

PipelineModel The fitted result of `Pipeline.fit()`. A Transformer that applies every fitted stage in order.

Predicate pushdown An optimization where a query’s filter is evaluated against column statistics (min/max per row group) before the actual data is read. Skips entire row groups whose statistics rule them out.

Probability simplex The set of vectors with non-negative entries that sum to 1. The output of softmax lives in the probability simplex.

Sentence-BERT A fine-tuning of BERT in a siamese architecture for sentence-level cosine similarity. The `sentence-transformers` library distributes pretrained checkpoints.

Silhouette score A clustering quality metric. Per-point silhouette ranges from -1 to $+1$; the corpus-level silhouette is the mean.

Snappy A fast mid-compression codec used as the default compression algorithm in Parquet. Good speed-vs-size tradeoff.

SODA Socrata Open Data API. The HTTP API used by NYC OpenData to serve the 311 corpus.

Softmax The normalization function that turns a vector of logits into a probability distribution. $\text{softmax}(x)_i = \exp(x_i) / \sum_j \exp(x_j)$.

Spark MLlib The machine-learning library distributed with Apache Spark. Provides distributed implementations of TF-IDF, Logistic Regression, Linear Regression, Word2Vec, K-Means, LDA, and more.

Spark Structured Streaming Spark’s API for incremental processing of unbounded data. Uses a micro-batch execution model with exactly-once semantics.

Stratified sampling Sampling that preserves the per-class proportions of the source distribution. Spark’s `sampleBy` implements stratified sampling via per-class fractions.

Stopword A high-frequency word (“the,” “and,” “of”) that is dropped during preprocessing because it does not discriminate between categories.

TF-IDF Term Frequency-Inverse Document Frequency. The product $\text{tf}(t, d) \cdot \text{idf}(t)$.

Transformer (Spark MLlib) A Spark abstraction for anything that takes a `DataFrame` and returns a `DataFrame` with new columns added. Not to be confused with the transformer architecture in deep learning.

Trigger.AvailableNow A Spark Structured Streaming trigger mode that processes every input that exists at start time and then stops. Useful for batch-as-stream demos.

Word2Vec A neural word embedding learned from a co-occurrence objective. The skip-gram variant predicts context from a center word.

References

Bibliography

- [1] Kontokosta, C.E. & Hong, B. (2021) Bias in 311 Reporting: Engagement and Inequality in the Use of Civic Service Requests. *Computers, Environment and Urban Systems* **87**.
- [2] Spärck Jones, K. (1972) A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation* **28**(1):11-21.
- [3] Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., et al. (2016) MLlib: Machine Learning in Apache Spark. *Journal of Machine Learning Research* **17**(34):1-7.
- [4] Mikolov, T., Chen, K., Corrado, G. & Dean, J. (2013) Efficient Estimation of Word Representations in Vector Space. In *Workshop Track of the International Conference on Learning Representations (ICLR)*.
- [5] Reimers, N. & Gurevych, I. (2019) Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [6] Wang, W., Wei, F., Dong, L., Bao, H., Yang, N. & Zhou, M. (2020) MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*.
- [7] Blei, D.M., Ng, A.Y. & Jordan, M.I. (2003) Latent Dirichlet Allocation. *Journal of Machine Learning Research* **3**:993-1022.
- [8] Zaharia, M., Chowdhury, M., Franklin, M.J., Shenker, S. & Stoica, I. (2010) Spark: Cluster Computing with Working Sets. In *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing (HotCloud)*.
- [9] Armbrust, M., Xin, R.S., Lian, C., Huai, Y., Liu, D., Bradley, J.K., et al. (2015) Spark SQL: Relational Data Processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*.
- [10] Karau, H., Wendell, P. & Fleming, M. (2017) *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. O'Reilly Media.
- [11] Appleby, A. (2008) MurmurHash3 algorithm. Public-domain non-cryptographic hash function, <https://github.com/aappleby/smhasher>.
- [12] PySpark Documentation, Apache Software Foundation. <https://spark.apache.org/docs/latest/api/python/>.

Bibliography

- [13] NYC OpenData, 311 Service Requests from 2010 to Present (2020+ endpoint, dataset id erm2-nwe9). <https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/erm2-nwe9>.
- [14] NYC OpenData, 311 Service Requests from 2010 to Present (2010-2019 historical endpoint, dataset id 76ig-c548). <https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/76ig-c548>.
- [15] Streamlit Documentation. <https://docs.streamlit.io/>.

Index

- ACID transactions, 15
- Apache Spark, 15
- architecture lesson, 73
- auto-commit, 7
- auto-commit graceful no-op, 57
- baked-in fallback, 57
- baseline discipline, 73
- Bias Audit tab, 67
- borough fingerprints, 48
- City Pulse tab, 67
- Cluster Atlas tab, 67
- community districts, 7
- Databricks, 7
- Delta Lake, 14
- embedding complementarity, 73
- Estimator, 15
- future work, 73
- gensim, 7
- Google Colab, 7
- honest failure framing, 73
- JVM stale state, 7
- K-Means, 11
- Kontokosta and Hong, 62
- Lambda architecture, 20
- Live Pulse tab, 67
- log1p, 10
- logistic regression
 - multinomial, 10
- macro-F1, 48
- MiniLM, 12
- MiniLM probe failure, 48
- ML Pipeline, 12
- mmh3 bridge, 57
- Murmurhash3, 7, 14
- Noise Street Sidewalk, 62
- numpy ABI, 7
- NYC 311 corpus, 7
- Parquet, 14
- parquet ancient dates, 7
- per-page checkpoints, 57
- Phase 0
 - environment, 37
- Phase 1
 - SODA ingest, 37
- Phase 10
 - MiniLM, 37
- Phase 2
 - preprocessing, 37
- Phase 3
 - classifier, 37
- Phase 4
 - regressor, 37
- Phase 5
 - Word2Vec, 37
- Phase 6
 - borough fingerprints, 37
- Phase 7
 - dashboard, 37
- Phase 8
 - streaming, 37
- Phase 9
 - LDA, 37
- Pipeline Status tab, 67
- PipelineModel, 15
- Plotly Dash, 7
- portable artifacts, 20
- predicate pushdown, 15
- regression
 - heavy-tailed, 10
- regressor lift, 48

Index

reporting bias, 62
resume gates, 57

sample size tradeoff, 62
Sentence-BERT, 12
silhouette, 48
silhouette score, 11
skip-gram, 11
SLA-bounded categories, 62
Snappy compression, 15
SODA endpoint, 7
Spark, 7
Spark local mode, 20
Spark MLlib, 12
Spark Structured Streaming, 13
stale JVM session, 57
Streamlit, 7
Streamlit Cloud, 20

TF-IDF, 9
Transformer, 15
Triage Bot tab, 67

Word2Vec, 11